

ArrowFlow: Hierarchical Machine Learning in the Space of Permutations

Ozgur Yilmaz*

Department of Artificial Intelligence

Adana Science and Technology University, Adana, Turkey

Abstract

We introduce **ArrowFlow**, a machine learning architecture whose core layers operate entirely in the space of permutations (real-valued inputs are first encoded into permutations by a preprocessing pipeline). Its computational units are *ranking filters*—learned orderings that compare inputs via Spearman’s footrule distance and update through permutation-matrix accumulation, a non-gradient rule rooted in displacement evidence. Layers compose hierarchically: each layer’s output ranking becomes the next layer’s input, enabling deep ordinal representation learning without any floating-point parameters in the core computation.

An encoding pipeline (polynomial expansion, random projection, argsort) bridges real-valued data and the permutation world, while a multi-view ensemble—-independent networks trained on diverse projections, combined by majority vote—compensates for information lost in the ordinal encoding. We connect the architecture to Arrow’s impossibility theorem: the rank aggregation performed by its learning rule provably violates the independence-of-irrelevant-alternatives axiom, and we argue that such social-choice non-neutrality (context dependence, specialization, symmetry breaking) acts as an inductive bias for nonlinearity, sparsity, and stability.

Experiments span UCI tabular benchmarks, MNIST, gene expression cancer classification (TCGA), and preference data, all against GridSearchCV-tuned baselines. ArrowFlow matches the best tuned baselines on Iris (2.7% vs. 3.3%; a sub-sample margin, and its configuration was selected on the test split—a proof-of-concept rather than a fair head-to-head, see protocol) and is competitive on most UCI datasets. Against a nearest-neighbor classifier on the *same* encoded permutations, the learned filters cut error by 2.5–12.5×, evidence that the learning rule contributes beyond the encoding. On gene expression data, rank-based methods are *exactly invariant* to strictly monotone within-sample transformations (stable at 2.5% where SVM on raw values collapses to 82.6%)—a property of the ordinal representation that ArrowFlow embodies architecturally rather than a unique advantage of the method. On MNIST via PCA, ArrowFlow reaches 9.1% error using only ordinal comparisons, with strong scaling as network width and depth increase. A single parameter—polynomial degree—acts as a master switch: degree 1 yields noise robustness (8–28% less degradation), rank-only

*ozguryilmaz@atu.edu.tr; <https://scholar.google.com/citations?user=AIJWYCAAAAAAJ>

(magnitude-hiding) robustness (+0.5pp cost), and missing-feature resilience; higher degrees trade these for improved clean accuracy.

ArrowFlow is not designed to surpass gradient-based methods. It is an existence proof that competitive classification is possible in a fundamentally different computational paradigm—one that elevates ordinal structure to a first-class citizen, with natural alignment to integer-only and neuromorphic hardware.

Code: <https://github.com/yilmazozgur/arrowflow>

1 Introduction

The dominant paradigm in modern machine learning represents data as real-valued tensors and learns transformations through gradient descent on differentiable, continuous parameter spaces. While extraordinarily successful, this paradigm has fundamental limitations when the underlying structure of data is *ordinal* or *relational* rather than metric. Consider classifying sequences by their ordering pattern: the sequence $[1, 2, 3, 4, 5]$ is “ascending” and $[5, 4, 3, 2, 1]$ is “descending.” The essential information is not the magnitude of elements but their *relative ordering*—a combinatorial, not metric, property.

ArrowFlow is built on the premise that **the fundamental data structure in a neural network can be a sorted list, and the fundamental operation can be a rank distance between two such lists**. In this framework:

- **Data points** are ordered sequences of tokens (permutations of a vocabulary).
- **Learned filters** are also permutations of the same vocabulary.
- **Similarity** is measured by Spearman’s footrule distance—the sum of absolute positional displacements between two orderings [Diaconis and Graham, 1977].
- **Learning** proceeds by re-ordering filter elements based on accumulated displacement evidence, rather than by adjusting floating-point weights.

This architecture draws from a productive tension in social choice theory. Arrow’s impossibility theorem [Arrow, 1951] demonstrates that any aggregator of individual rankings must violate at least one fairness axiom (Pareto efficiency, independence of irrelevant alternatives, non-dictatorship) on some preference profiles. In learning, we *exploit* these violations as inductive biases: context dependence (IIA violations) yields nonlinearity; specialization (ND violations) yields sparsity and winner-take-all dynamics; partial monotonicity (Pareto-like behavior) stabilizes training. The name “ArrowFlow” reflects both this connection to Arrow’s theorem and the flow of ordering information through the network layers.

What ArrowFlow is—and is not. ArrowFlow is not designed to be the most accurate or fastest classifier. It is a *fundamentally different computational paradigm*: a hierarchical, filter-based machine learning system whose core layers operate entirely in the space of permutations (with a real-valued encoder only at the input). The contribution is the paradigm itself—showing that competitive classification is achievable without any floating-point parameters in the core computation, and that ordinal architectures possess structural

advantages (noise robustness, rank-only/magnitude-hiding operation, graceful handling of missing data) that are not automatic in conventional methods unless rank transformations or robust preprocessing are added.

Contributions.

1. A formal definition of the *ranking layer* with displacement motions, permutation-matrix accumulation, and hierarchical rank aggregation (Section 3).
2. The multi-view ensemble architecture with diverse projection strategies that addresses the argsort encoding bottleneck and achieves competitive accuracy (Sections 5–6).
3. An Arrow-theoretic interpretation of layer expressivity, connecting fairness-axiom violations to inductive biases for nonlinearity, sparsity, and stability (Section 4).
4. Formal theoretical analysis (Section 8):
 - an argsort stability theorem with Gaussian tail bound, and a projected-noise corollary, explaining the noise robustness;
 - an information-capacity theorem ($\log_2(e!)$ bits for an e -dimensional score vector, via permutation cones);
 - a polynomial noise-amplification (local sensitivity) result for the accuracy–robustness trade-off;
 - a consistency proof for the accumulation rule—a Borda/mean-position estimator—and an ensemble error bound; and
 - a social-choice subsection characterizing filter learning as consistent rank aggregation (with exponential consistency), proving two-sided adversarial-perturbation bounds from the footrule metric, and showing that the learning rule’s positional aggregation provably violates Arrow’s independence axiom.
5. Comprehensive experiments across UCI tabular benchmarks, MNIST (via PCA, isolating the sort-layer classifier), gene expression cancer classification (TCGA), and the Sushi preference dataset. These reveal a unique robustness profile governed by the polynomial degree parameter, exact invariance to within-sample strictly monotone distribution shifts, and architecture scaling that has not plateaued at 1024 filters (Sections 10–11).
6. A quantitative energy analysis showing that ArrowFlow’s integer-only arithmetic is $15\times$ cheaper *per layer in arithmetic energy* than equivalent FP32 MLP layers (narrowing to $\approx 1.05\times$ at full seven-view inference, with larger gains expected from reduced memory traffic and multiplier-free hardware), with natural alignment to neuromorphic hardware and sorting-network implementations (Section 12.3).

Note. This is an extended technical report presenting the full theory, experiments, and discussion for ArrowFlow.

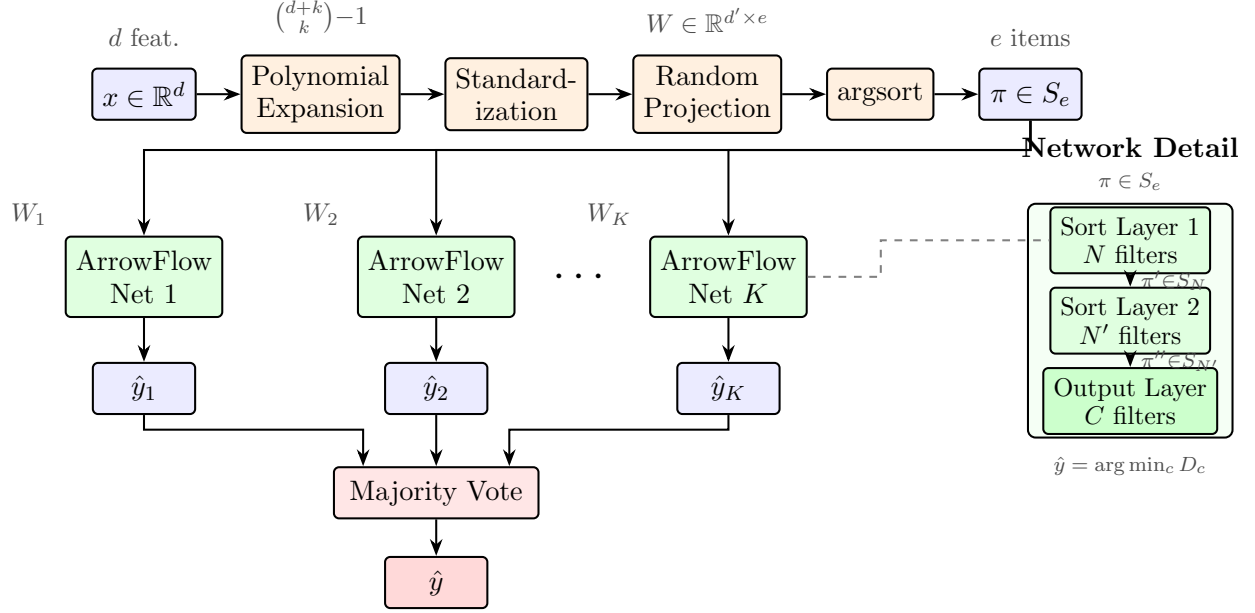


Figure 1: **ArrowFlow pipeline.** Real-valued input x is encoded via polynomial expansion, standardization, random projection, and argsort. K independent networks receive different projections W_k . Each network (detail, right) is a stack of sort layers computing Spearman’s footrule distance to learned filters. Predictions are combined by majority vote.

2 Related Work

ArrowFlow sits at the intersection of several research traditions: neuroscience-inspired rank-order coding, differentiable sorting, permutation distance metrics, non-gradient learning, social choice theory, random projections, and ensemble methods. We survey each in turn, positioning ArrowFlow’s contributions relative to prior work.

2.1 Rank Order Coding

Rank Order Coding (ROC) encodes stimulus strength via the *order* of neural spikes rather than absolute firing rates, highlighting order as an information carrier [Thorpe and Gautrais, 1998, Gautrais and Thorpe, 1998]. This biological principle—that relative ordering is more robust and efficient than precise magnitude encoding—motivates ArrowFlow’s core design. The approach has shown promise in image recognition, where relative pixel intensities can be more informative than precise values under varying conditions [Jost et al., 2002, Bonilla et al., 2022]. ArrowFlow generalizes ROC from a fixed encoding scheme to a *learnable hierarchical transformation* of orderings across layers.

2.2 Differentiable Sorting and Sinkhorn Networks

Recent work has made sorting operations differentiable for end-to-end learning. NeuralSort [Grover et al., 2019] and SoftSort [Prillo and Eisenschlos, 2020] provide continuous relax-

ations of argsort. Blondel et al. [Blondel et al., 2020] introduced fast differentiable sorting via optimal transport. Gumbel-Sinkhorn networks [Mena et al., 2018] learn latent permutations through doubly stochastic matrices, leveraging the classical Sinkhorn normalization [Sinkhorn and Knopp, 1967]. These approaches treat order as a soft surrogate for continuous scores—they relax permutations everywhere to enable gradient flow. ArrowFlow takes the opposite approach: it **operates directly on discrete permutations** and replaces gradient descent with permutation-matrix-based updates. Differentiability is needed only at the boundaries between rank and tensor layers, not within the ranking computation itself.

2.3 Permutation Distance Metrics

Kendall [Kendall, 1938] introduced the tau rank correlation coefficient, counting pairwise disagreements. Diaconis and Graham [Diaconis and Graham, 1977] established Spearman’s footrule as a closely related metric, proving that footrule and Kendall tau distance are within a constant factor. ArrowFlow uses Spearman’s footrule (ℓ_1 norm of positional displacements) as its core distance. Fagin et al. [Fagin et al., 2003] extended these metrics to partial rankings, relevant to ArrowFlow’s handling of receptive fields and variable-length inputs.

2.4 Non-Gradient Learning

ArrowFlow’s sort layers do not use gradient descent. Its closest relative is *Learning Vector Quantization* (LVQ) and Kohonen’s Self-Organizing Maps [Kohonen, 1990], which learn by competitive prototype updates—the nearest prototype is nudged toward (LVQ also: away from) each labeled input. ArrowFlow’s output layer is precisely this attractive, error-driven prototype rule, and its hidden layers add the LVQ-style repulsive direction (Section 7.2), with two differences from classical LVQ: the prototypes are *permutations* updated by re-ordering rather than real-valued vectors updated by subtraction, and learning composes hierarchically across stacked layers. This competitive dynamic also relates to Grossberg’s Adaptive Resonance Theory [Grossberg, 1987], where winner-take-all prototypes are updated by matching and accumulation. Pointer Networks [Vinyals et al., 2015b] and the “Order Matters” framework [Vinyals et al., 2015a] demonstrated that input and output ordering significantly impacts sequence-to-sequence performance, motivating architectures that treat ordering as a learnable structure. More recently, Hinton’s Forward-Forward algorithm [Hinton, 2022] and Target Propagation [Lee et al., 2015] have revived interest in alternatives to backpropagation. ArrowFlow contributes to this line by demonstrating that competitive, displacement-based learning can work in discrete permutation spaces.

2.5 Social Choice Theory and Rank Aggregation

Arrow’s impossibility theorem [Arrow, 1951] demonstrates unavoidable trade-offs in rank aggregation—no mechanism aggregating three or more alternatives can simultaneously satisfy Pareto efficiency, independence of irrelevant alternatives, and non-dictatorship. Rank aggregation has been extensively studied in information retrieval, where Dwork et al. [Dwork et al., 2001] proposed combining ranked results from multiple search engines using median-based methods on Kendall tau distance. The Mallows model [Mallows, 1957] provides a

probabilistic framework for permutations centered on a modal ranking, parameterized by a dispersion parameter. Conitzer and Sandholm [Conitzer and Sandholm, 2005] showed that common voting rules (Borda, Kemeny) correspond to maximum likelihood estimation under different noise models over permutations, providing a statistical-learning interpretation of the social-choice concepts that ArrowFlow operationalizes. Rank-Ordered Autoencoders [Bertens, 2016] use rank-preserving embeddings and sparsity constraints. ArrowFlow generalizes from *preserving* rank to *transforming* rank across layers, and uniquely reinterprets Arrow’s fairness-axiom violations as inductive biases for nonlinearity, sparsity, and stability.

2.6 Random Projections and Dimensionality Reduction

ArrowFlow’s encoding pipeline relies on random projection to map data into a space where argsort produces informative permutations. The Johnson-Lindenstrauss lemma [Johnson and Lindenstrauss, 1984] guarantees that random projections approximately preserve pairwise distances. Achlioptas [Achlioptas, 2003] showed that even very sparse random matrices suffice for this purpose. Arriaga and Vempala [Arriaga and Vempala, 2006] proved that random projections preserve margin-based learnability, directly justifying ArrowFlow’s pipeline of random projection followed by classification. Cannings and Samworth [Cannings and Samworth, 2017] proposed random-projection ensemble classification, combining classifiers trained on different random projections via data-driven voting—ArrowFlow’s multi-view ensemble is a permutation-space instance of exactly this framework.

2.7 Ensemble Methods and Multi-View Learning

ArrowFlow’s multi-view ensemble draws on classical ensemble theory. Breiman’s bagging [Breiman, 1996] established variance reduction through bootstrap aggregation. Schapire [Schapire, 1990] proved that any weak learner can be boosted to arbitrary accuracy, providing theoretical grounding for why ArrowFlow’s individual views, even if weak, combine into a strong ensemble. Dietterich [Dietterich, 2000] showed that ensemble diversity is as important as individual accuracy. Condorcet’s jury theorem provides the theoretical foundation: if each voter has error rate $p < 0.5$ and errors are independent, the majority-vote error decreases exponentially with the number of voters. ArrowFlow maximizes the independence condition by using diverse projection strategies—each view sees a different ordinal representation of the same data. This connects to the multi-view learning framework of Xu et al. [Xu et al., 2013], where complementary “views” are fused for improved prediction.

2.8 Robustness Through Discretization

A distinctive finding of ArrowFlow is that the argsort discretization provides measurable noise robustness when polynomial expansion is absent. This connects to a broader literature on discretization for robustness. Quantization-aware training [Jacob et al., 2018] shows that reduced-precision representations can improve robustness. The VQ-VAE [van den Oord et al., 2017] demonstrated that discretizing latent representations via vector quantization improves generalization and avoids posterior collapse—ArrowFlow’s argsort is an extreme form of discretization that converts continuous features to combinatorial objects. More directly,

the rank transform [Conover and Iman, 1981] has long been used in nonparametric statistics precisely because it is immune to monotone transformations and outliers. ArrowFlow’s encoding pipeline makes this classical statistical insight into an architectural principle.

2.9 Learning to Rank

ArrowFlow also connects to the learning-to-rank literature in information retrieval, where the goal is to learn a function that orders items by relevance. RankNet [Burgess et al., 2005] uses gradient descent on pairwise ranking losses; LambdaMART [Burgess, 2010] combines boosted trees with ranking-specific objectives. These methods learn continuous scoring functions whose *output* is a ranking. ArrowFlow inverts this relationship: both the *input* and the *learned parameters* are rankings, and learning operates by re-ordering filters rather than adjusting scores. This makes ArrowFlow more akin to a nearest-neighbor method in permutation metric space, enhanced with hierarchical learned embeddings.

3 The Ranking Layer

The ranking layer is the fundamental computational unit of ArrowFlow. It operates in the space of orderings rather than numeric activations. Each layer transforms an input ranking into a new ranking representation by comparing it to a bank of learned *ranking filters*. We use the terms *ranking layer* and *sort layer* interchangeably throughout.

3.1 Notation

Let $\mathcal{V} = \{1, \dots, V\}$ denote a vocabulary of tokens. A ranking is a permutation $\pi \in S_V$. We use two complementary maps and keep them distinct throughout:

$$\text{item}_\pi(p) = \text{the token at position } p, \quad \text{pos}_\pi(v) = \text{the position of token } v.$$

Worked examples list tokens by position—e.g. $\pi_x = [C, A, E, B, D]$ means $\text{item}_{\pi_x}(1) = C$ —while the displacement formulas below use pos . We write $\text{argsort}(y)$ for the permutation that lists the entries of $y \in \mathbb{R}^V$ in *increasing* order (matching NumPy’s convention and the implementation); thus when the layer ranks filters by distance, the smallest distance—the closest filter—appears first. Where a decreasing order is more natural (the permutation cones of Theorem 7) we write $\text{argsort}_\downarrow$; the two differ only by reversal. We abbreviate $\text{rank}(r, v) = \text{pos}_r(v)$ for the position of token v in a filter r ’s ordering.

3.2 Ranking Filters and Displacement Motions

A ranking layer F^{rank} contains N ranking filters $\{r_j\}_{j=1}^N$, each storing a learned local ordering over the vocabulary. Given an input ranking π_x , the layer computes a *motion* vector (a signed positional displacement, not a string-edit operation) $m_j \in \mathbb{Z}^\ell$ for each filter:

$$m_j[p] = \text{rank}(r_j, \pi_x[p]) - p, \quad p = 1, \dots, \ell. \tag{1}$$

The motion $m_j[p]$ measures how many positions element $\pi_x[p]$ must move to reach its position in filter r_j . This is a signed displacement—positive means the element should move toward the end, negative toward the front.

A distance is computed via the ℓ_q norm:

$$D_j = \|m_j\|_q, \quad q \in \{1, 2\} \text{ (with an optional } \ell_0\text{-style count, } q = 0, \text{ which is a pseudonorm).} \quad (2)$$

With $q = 1$, this is Spearman’s footrule distance [Diaconis and Graham, 1977]: the total positional displacement between the input and the filter. All experiments in this paper use $q = 1$.

Missing data and receptive fields. If π_x omits items present in r_j , the experiments in this paper assign each missing item the central position mV (with $m = 0.5$) and down-weight its update contribution by $w = 0.5$ (Section 7.2); an alternative variant instead displaces missing items to the end of the list and charges a deletion penalty. We use the central-imputation rule throughout unless stated otherwise. If r_j has a restricted receptive field (subset of items), the average motion is subtracted to promote translation invariance—analogueous to zero-centering in convolutional layers.

3.3 Forward Pass: From Distances to Output Ranking

The layer aggregates N filter responses into a new ranking. The simplest and most effective aggregation is:

$$\pi' = \text{argsort}(D_1, D_2, \dots, D_N), \quad (3)$$

which ranks filters from closest (smallest distance, most similar) to farthest. The output π' is a permutation of filter indices—a new sorted list in a vocabulary of size N —and becomes the input to the next layer.

This is closer to a prototype (radial-basis) layer than to a convolution: each filter is a learned permutation prototype, the layer scores the input by rank distance to every prototype, and the resulting ranking of prototypes defines the next representation. (We reserve the convolution analogy for the receptive-field variant, where a filter sees only a subset of items.)

3.4 Backward Pass: Motion as Discrete Gradient

The key insight is that the motion vector m_j from Eq. (1) plays the role of a gradient in continuous optimization—it tells us *how to move each element* to reduce distance.

Proposition 1 (Motion as Discrete Gradient). *Because the footrule distance $d_F(\pi_x, r_j) = \sum_{p=1}^{\ell} |m_j[p]|$ decomposes into independent per-position terms, its minimization over per-item integer shifts $\{\delta_p\}$ separates across positions, and the unique minimizer of each term is the motion of Eq. (1):*

$$m_j[p] = \arg \min_{\delta \in \mathbb{Z}} |\text{rank}(r_j, \pi_x[p]) - (p + \delta)|, \quad p = 1, \dots, \ell.$$

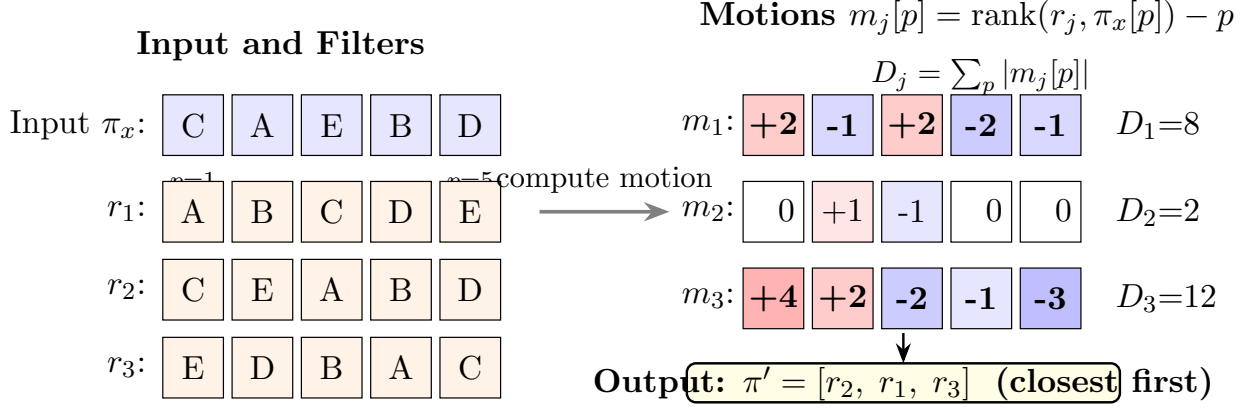


Figure 2: **Ranking layer forward pass.** Input $\pi_x = [C, A, E, B, D]$ is compared to three filters. The motion $m_j[p] = \text{rank}(r_j, \pi_x[p]) - p$ measures signed displacement (red = forward, blue = backward). The footrule distance $D_j = \sum |m_j[p]|$ totals the displacements. Filters are ranked by proximity: r_2 (distance 2) is closest. The output π' becomes the input to the next layer.

Hence the full motion vector m_j is the exact per-item descent direction for the footrule objective under an independent-shift relaxation (the simultaneous shifts need not compose into a valid permutation).

The motion vector thus points each item directly toward its target position in the filter, the discrete analogue of a negative loss gradient—no calculus is involved, and the update signal is simply the motion between the input ordering and the correct-class filter. The separability exploited here is precisely what distinguishes footrule from non-decomposable metrics such as Kendall tau: it is what lets the accumulation rule of Section 3.5 treat each item’s displacement independently. (We caution that summing the per-item minimizers does not itself yield a permutation; the motion is a descent *direction*, realized as a valid ordering only after the accumulation-and-reorder projection of Eqs. 4–5.)

3.5 Permutation-Matrix Accumulation

Each filter r_j maintains an accumulator $A_j \in \mathbb{R}^{V \times V}$ (initialized to identity) that integrates displacement votes across training examples. Given input π_x , the *vote matrix* $\Phi(\pi_x, r_j) \in \{0, 1\}^{V \times V}$ records where each item appears in the input:

Definition 2 (Vote Matrix). *Each item v in the vocabulary votes for its position in the input π_x :*

$$\Phi(\pi_x, r_j)[v, k] = \begin{cases} 1 & \text{if } k = \text{rank}(\pi_x, v), \\ 0 & \text{otherwise,} \end{cases}$$

where $\text{rank}(\pi_x, v)$ denotes the position of item v in the input ordering π_x . Each row of Φ has exactly one nonzero entry: item v votes for the position that the training data places it at.

Equivalently, for each filter position p , the backward motion $m_j^{\text{bwd}}[p] = \text{rank}(\pi_x, r_j[p]) - p$ measures where filter item $r_j[p]$ appears in the input, and the vote places $r_j[p]$ at position $p + m_j^{\text{bwd}}[p]$. Note that this backward motion is the negative of the forward motion from Eq. (1): the forward motion measures displacement *from input to filter*, while the backward motion measures displacement *from filter to input*, providing the update direction.

The accumulator is updated additively:

$$A_j \leftarrow A_j + \Phi(\pi_x, r_j). \quad (4)$$

After T updates, $A_j[v, k]$ counts the total evidence that item v belongs at position k . The filter’s ordering is recomputed via weighted index averaging:

$$\hat{i}(v) = \frac{\sum_k k \cdot A_j[v, k]}{\sum_k A_j[v, k]}, \quad r_j \leftarrow \text{argsort}(\{\hat{i}(v)\}_{v \in \mathcal{V}}). \quad (5)$$

This resembles a projection onto the set of permutations: accumulate displacement evidence, then map to the ordering consistent with the aggregate evidence (we do not claim a formal proximal operator). The accumulator smooths individual motions and provides a form of momentum.¹

Remark 1. *The learning rule has no learning rate in the continuous sense. Instead, the accumulator’s history provides natural damping: recent motions contribute proportionally to the total accumulated evidence. An explicit learning rate η can optionally scale the vote matrix Φ to control update speed.*

3.6 Training Algorithm

4 Arrow’s Impossibility Theorem as Design Principle

Each ranking layer functions as a *micro-society* of filters aggregating preferences. Arrow’s impossibility theorem states that no social choice function over three or more alternatives can simultaneously satisfy:

- **Pareto Efficiency (PE):** if all filters prefer $a \succ b$, the output preserves $a \succ b$.
- **Independence of Irrelevant Alternatives (IIA):** the relative output ranking of a vs. b depends only on filters’ rankings of a vs. b , not on c .
- **Non-Dictatorship (ND):** no single filter determines the output for all inputs.

In ArrowFlow, these constraints become *capacity knobs*:

¹Eq. (5) normalizes each item’s score by its row sum $\sum_k A_j[v, k]$; the implementation normalizes by the corresponding column sum. The two coincide (and the analysis of Proposition 9 applies exactly) whenever the accumulated votes form a permutation matrix, i.e. absent the position clipping and missing-item handling used for partial inputs.

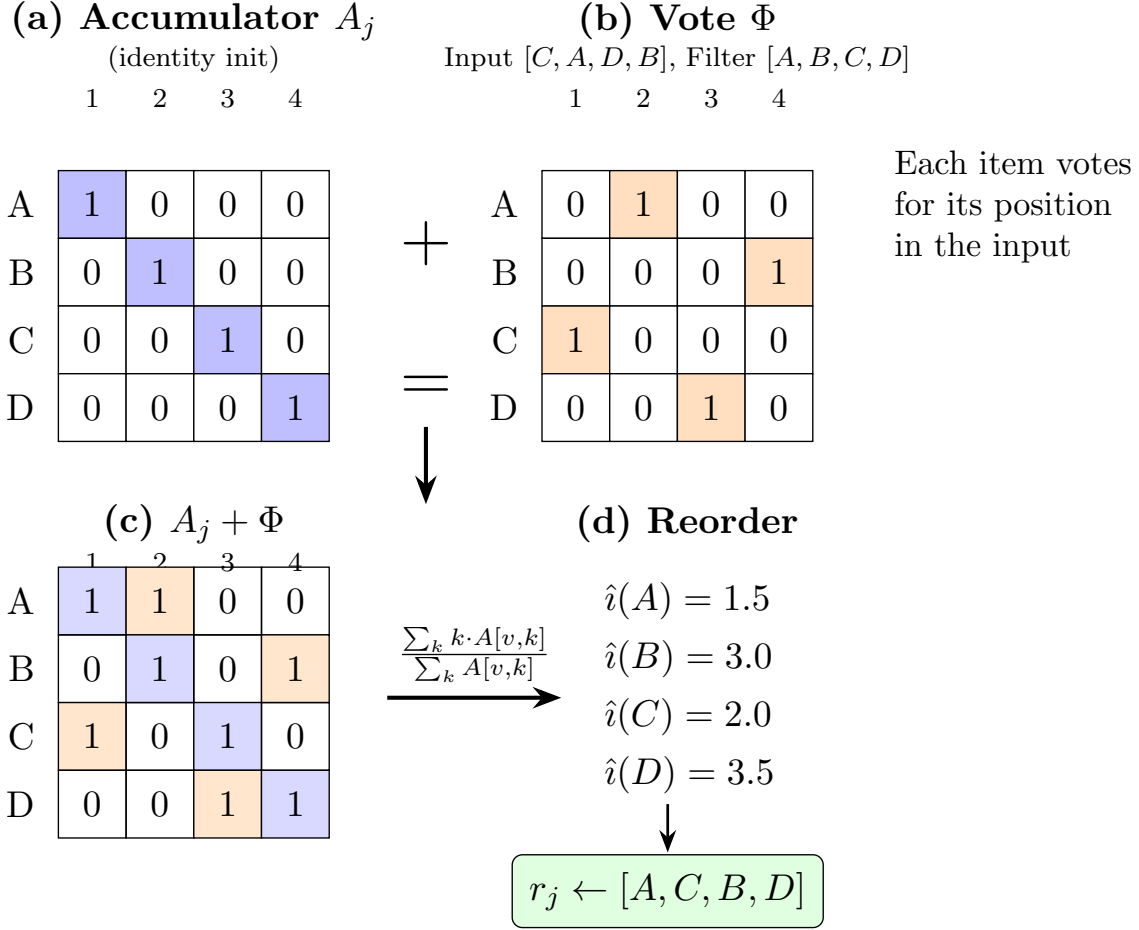


Figure 3: **Permutation-matrix accumulation.** (a) Accumulator starts as identity (encoding current filter $[A, B, C, D]$). (b) Training input $[C, A, D, B]$: each item votes for its position in the input (Def. 2). (c) Votes sum additively (Eq. 4). (d) Weighted average and argsort yield the updated filter $[A, C, B, D]$ (Eq. 5), which has moved toward the training data. Convergence to the mean-position ranking is guaranteed (Proposition 9).

PE $\rightarrow$ Stability and gradient propagation. When filters unanimously prefer $a \succ b$, motions preserve this ordering, yielding a stabilizing monotonicity. This is analogous to residual connections—unanimous evidence propagates without attenuation.

IIA and context dependence. A single layer’s *forward* pass orders filters by their independent distances, so it respects pairwise IIA; its nonlinearity comes from the piecewise-constant cone geometry of argsort (Theorem 7), the permutation-space counterpart of an activation function. The IIA violation that Arrow’s theorem guarantees appears instead in the *learning rule*, whose positional aggregation of training rankings is context-dependent (Proposition 15): an item’s consensus position shifts with the placement of others. This is what shapes filters into context-sensitive detectors rather than pairwise-separable templates; the formal statement is deferred to Section 8.7.

Algorithm 1 Ranking Layer Forward–Backward

Require: Input ranking π_x , filters $\{r_j\}_{j=1}^N$, accumulators $\{A_j\}$

- 1: **Forward:**
 - 2: **for** $j = 1$ to N **do**
 - 3: $m_j \leftarrow \text{Motion}(\pi_x, r_j)$ ▷ signed positional displacements, Eq. (1)
 - 4: $D_j \leftarrow \|m_j\|_1$ ▷ Spearman’s footrule
 - 5: **end for**
 - 6: $\pi' \leftarrow \text{argsort}(D_1, \dots, D_N)$ ▷ rank filters by proximity (closest first)
 - 7: **return** π'

 - 8: **Backward (supervised update):** see Section 7.2 and Algorithm 2: the target ranking selected by the label (output layer) or propagated from above (hidden layers) is turned into a vote matrix Φ (Def. 2), accumulated into A_j (Eqs. 4–5), and the filter reordered.
-

ND violation → Sparse specialization. When a few filters consistently have the smallest distances for certain input types, they dominate the output ranking—a winner-take-all dynamic analogous to attention heads or sparse mixture-of-experts. This creates feature dominance, aiding generalization and efficiency.

Depth as compositional rank refinement. Stacking ranking layers enables compositional processing:

- Early layers learn local tournaments or pairwise motifs (e.g., Condorcet-like cycles, single-peaked segments).
- Middle layers learn contextual re-weighting (via the context-dependent aggregation of Proposition 15) to resolve conflicts.
- Late layers enforce global consistency and task-specific signals.

This is analogous to ConvNets (local filters → mid-level patterns → global structure), but in the permutation domain.

5 Encoding: From Real-Valued Data to Permutations

The central challenge in applying ArrowFlow to real-valued data is the *encoding problem*: converting continuous feature vectors into permutations without catastrophic information loss. The argsort operation that converts a real vector to a ranking discards all magnitude information, preserving only relative order. Two vectors $[1.0, 2.0, 3.0]$ and $[0.01, 100, 100.01]$ produce the same ranking $[1, 2, 3]$ despite being vastly different.

5.1 Polynomial Feature Expansion

For low-dimensional data, we first expand the feature space via polynomial features:

$$x \in \mathbb{R}^d \mapsto \phi(x) \in \mathbb{R}^{\binom{d+k}{k}-1}, \quad (6)$$

where k is the polynomial degree. For $d = 4$ (Iris) at degree 3, this expands from 4 to 34 features, creating interaction terms $x_i x_j$, $x_i x_j x_k$ that dramatically increase the diversity of achievable rankings. This expansion is critical: on Iris, polynomial expansion alone reduces error by about $1.5\times$ (21.3%→14.7%; Table 10), and contributes to a larger reduction in combination with the multi-view ensemble.

5.2 Random Projection and Argsort

After optional polynomial expansion and standardization, features are projected to a target embedding dimension e via a random projection matrix $W \in \mathbb{R}^{d' \times e}$:

$$z = x \cdot W, \quad \pi = \text{argsort}(z). \quad (7)$$

The resulting permutation π is the input to the first sort layer. Different random matrices W produce different permutations from the same input—the key source of ensemble diversity.

5.3 Target-Aware Projection

To inject supervised signal, we construct *target-aware* projections that mix Linear Discriminant Analysis (LDA) components with random components:

$$W_{\text{aware}} = [W_{\text{LDA}} \quad W_{\text{random}}], \quad (8)$$

where W_{LDA} captures the most class-discriminative directions and W_{random} provides diversity. The LDA ratio controls the balance.

6 Multi-View Ensemble Architecture

The most impactful architectural innovation in ArrowFlow is the multi-view ensemble. A single projection W produces a single “view” of the data in permutation space. Since different projections capture different ordinal relationships, training multiple independent networks on different projections and combining via majority vote dramatically reduces error.

6.1 Architecture

Given K views:

1. Generate K projection matrices $\{W_k\}_{k=1}^K$ using diverse strategies: *random* (Section 5), *target-aware* (LDA-augmented, Section 5), and *calibrated*—a random projection whose projected scores are standardized per dimension before argsort, so that every projected dimension contributes equally to the ranking rather than being dominated by high-variance directions.

2. For each view k : encode all data via W_k , train an independent ArrowFlow network.
3. Combine predictions via majority vote: $\hat{y} = \text{mode}(\hat{y}_1, \dots, \hat{y}_K)$.

6.2 Theoretical Foundation

By Condorcet’s jury theorem, if each view has error rate $p < 0.5$ and errors are independent, the ensemble error decreases exponentially with K (formalized in Theorem 10). Independence is maximized by diverse projection strategies: views cycle through target-aware, random, and calibrated projections. Empirically, 7 views provide the best accuracy–cost trade-off, reducing error by up to $\sim 3\times$ compared to a single view (dataset-dependent; Section 10).

6.3 Permutation Data Augmentation

To improve sort-layer generalization, each training permutation can be augmented by applying random adjacent transpositions—the minimal perturbation in Spearman footrule distance (exactly 2 units). This is the permutation analogue of Gaussian noise in Euclidean space, adapted to the discrete metric topology.

7 Network Architecture

7.1 Layer Structure

Each ArrowFlow network is a layered architecture where each layer is a **Sort Layer**: a bank of N permutation filters. The layer computes Spearman’s footrule distance between the input permutation and each filter, and outputs a ranking of filters by distance. Layers are stacked: the output vocabulary of layer ℓ (filter IDs) becomes the input vocabulary of layer $\ell + 1$.

The default architecture uses two sort layers: a hidden layer with N filters and an output layer with C filters (one per class). Classification is by nearest-filter: the predicted class is the index of the output-layer filter closest to the final hidden representation.

Layer dimensions and vocabularies. The vocabulary changes at every layer. The input is a permutation of the e encoded tokens; layer 1 holds N_1 filters, each a permutation of those e tokens, and outputs a ranking of its filters—a permutation in S_{N_1} . Layer 2’s filters are therefore permutations of the *filter indices* $\{1, \dots, N_1\}$, and so on. The output layer holds C filters (one per class), permutations of $\{1, \dots, N_L\}$, and classification reduces to selecting the nearest class filter.

Layer	input vocabulary	# filters	output
1	e	N_1	$\pi^{(1)} \in S_{N_1}$
2	N_1	N_2	$\pi^{(2)} \in S_{N_2}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
output	N_L	C	$\hat{y} = \arg \min_c D_c^{(L)}$

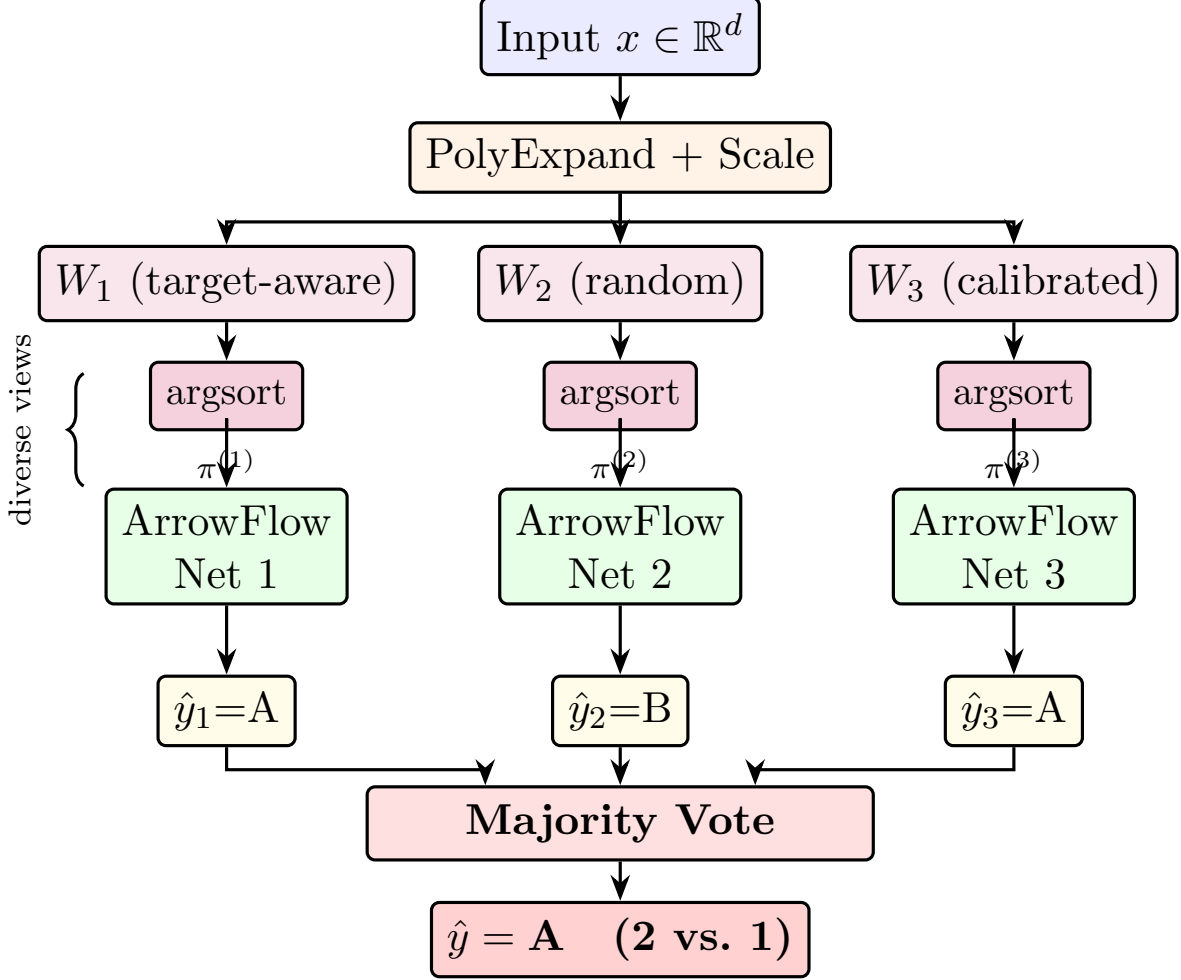


Figure 4: **Multi-view ensemble.** A single preprocessed input is projected through K different matrices using diverse strategies (target-aware, random, calibrated). Each produces a different permutation—a different ordinal “view.” Independent ArrowFlow networks are trained per view; predictions are combined by majority vote. Projection diversity approximates the independence condition of Theorem 10.

An item absent from an input (e.g. under feature masking) is assigned the central position mV with $m = 0.5$, so it contributes a moderate, bounded displacement rather than an extreme one; in filter updates its displacement is additionally down-weighted by the deletion-cost weight $w = 0.5$.

7.2 Supervised Training of Sort Layers

The forward pass (Section 3) maps an input permutation $\pi^{(0)} \in S_e$ through layers $\ell = 1, \dots, L$: layer ℓ computes $D_j^{(\ell)} = d_F(\pi^{(\ell-1)}, r_j^{(\ell)})$ for each filter and emits $\pi^{(\ell)} = \text{argsort}(D^{(\ell)})$ (closest filter first). The prediction is the nearest output filter, $\hat{y} = \arg \min_c D_c^{(L)}$. We now specify the complete supervised update, which is non-gradient and built entirely from the accumulation rule of Section 3.5.

Error-driven gating. On example (x, y) with prediction $\hat{y}$, define an update magnitude g : if $\hat{y} \neq y$ (a mistake) then $g = 1$; if $\hat{y} = y$ then $g = 1$ with probability p_c and $g = 0$ otherwise, with $p_c = 0.01$. Updates therefore concentrate on misclassified examples—in the spirit of perceptron and LVQ learning—while a small p_c keeps a trickle of reinforcement on already-correct ones.

Output layer: pull the correct-class filter. Only the class- y output filter is updated, and only toward the current hidden representation $\pi^{(L-1)}$:

$$A_y^{(L)} \leftarrow A_y^{(L)} + \frac{\eta g}{N_L} \Phi(\pi^{(L-1)}, r_y^{(L)}), \quad r_y^{(L)} \leftarrow \text{reorder}(A_y^{(L)}), \quad (9)$$

where Φ is the vote matrix (Definition 2) and reorder is the weighted-average argsort of Eq. (5). Every other output filter is left unchanged. At the output layer the update is purely *attractive*: the correct class’s prototype moves toward the input, and incorrect-class filters are never pushed away (the repulsion branch described below is used only in hidden layers, never at the output). The learning rate η (default 0.1) sets each example’s vote weight η/N_L relative to the unit-weight identity prior in the accumulator; it scales votes, not a gradient.

Hidden layers: motion propagation. The signed displacement field $\mu^{(L)}$ produced by the output update—how far each element of $\pi^{(L-1)}$ must move to reach $r_y^{(L)}$ —is the supervised signal handed to the layer below. For $\ell = L - 1$ down to 1, given the incoming target motion $\mu^{(\ell+1)}$:

1. **Top- ρ selection.** Order the layer’s filters by $|\mu^{(\ell+1)}|$ and update only the top ρN_ℓ of them ($\rho = 0.5$); the small-motion remainder is left untouched—a sparsity that both regularizes and halves the update cost.
2. **Signed accumulation.** Each selected filter $r_j^{(\ell)}$ accumulates votes with *nonnegative* weight $|\mu_j^{(\ell+1)}|$ toward a target $\tilde{\pi}$ whose choice is set by the *sign* of the propagated motion—the layer input $\pi^{(\ell-1)}$ when $\mu_j^{(\ell+1)} \geq 0$ (attractive), or its reversal $\text{rev}(\pi^{(\ell-1)})$ when $\mu_j^{(\ell+1)} < 0$ (repulsive):

$$A_j^{(\ell)} \leftarrow A_j^{(\ell)} + \frac{2\eta}{N_\ell} |\mu_j^{(\ell+1)}| \Phi(\tilde{\pi}, r_j^{(\ell)}), \quad r_j^{(\ell)} \leftarrow \text{reorder}(A_j^{(\ell)}), \quad (10)$$

with $\tilde{\pi} = \pi^{(\ell-1)}$ for $\mu_j^{(\ell+1)} \geq 0$ and $\tilde{\pi} = \text{rev}(\pi^{(\ell-1)})$ otherwise, as above. Because the weights $|\mu_j^{(\ell+1)}|$ are nonnegative, each $A_j^{(\ell)}$ remains a vote-count matrix, so the Borda/mean-position reading of Proposition 9 still applies (that proposition analyzes the accumulator mechanics; it does not model the sign-dependent choice of $\tilde{\pi}$, which is what makes the hidden update a signed, attract-or-repel rule).

3. **Cross-filter aggregation.** The motion passed further down is the *mean* over the layer’s filters of their per-item displacements, renormalized to peak magnitude $c N_\ell$ ($c = 0.125$) and sorted by amplitude:

$$\bar{\mu} = \frac{1}{N_\ell} \sum_j \mu_j^{(\ell)}, \quad \mu^{(\ell)} = \text{sort}_{|\cdot|} \left(c N_\ell \frac{\bar{\mu}}{\max_i |\bar{\mu}_i|} \right).$$

Algorithm 2 Supervised training step (one example, sort-only network)

Require: example (x, y) ; filters $\{r_j^{(\ell)}\}$, accumulators $\{A_j^{(\ell)}\}$; rate η , fraction ρ , scale c , correct-update prob. p_c

- 1: $\pi^{(0)} \leftarrow \text{encode}(x)$
- 2: **for** $\ell = 1$ to L **do** $\triangleright$ forward
- 3: $D_j^{(\ell)} \leftarrow d_F(\pi^{(\ell-1)}, r_j^{(\ell)}) \forall j$; $\pi^{(\ell)} \leftarrow \text{argsort}(D^{(\ell)})$
- 4: **end for**
- 5: $\hat{y} \leftarrow \arg \min_c D_c^{(L)}$
- 6: $g \leftarrow 1$ if $\hat{y} \neq y$; else $g \leftarrow 1$ w.p. p_c , else 0 $\triangleright$ error-driven gating
- 7: $\mu^{(L)} \leftarrow$ displacement of $\pi^{(L-1)}$ toward $r_y^{(L)}$ $\triangleright$ computed even if output frozen
- 8: **if** output layer not frozen **then**
- 9: $A_y^{(L)} += (\eta g / N_L) \Phi(\pi^{(L-1)}, r_y^{(L)})$; $r_y^{(L)} \leftarrow \text{reorder}(A_y^{(L)})$
- 10: **end if**
- 11: **for** $\ell = L - 1$ down to 1 **do** $\triangleright$ motion propagation
- 12: $S \leftarrow$ top ρN_ℓ filters by $|\mu^{(\ell+1)}|$
- 13: **for all** $j \in S$ **do**
- 14: $\tilde{\pi} \leftarrow \pi^{(\ell-1)}$ if $\mu_j^{(\ell+1)} \geq 0$ else $\text{rev}(\pi^{(\ell-1)})$ $\triangleright$ sign $\Rightarrow$ attract/repel
- 15: $A_j^{(\ell)} += (2\eta / N_\ell) |\mu_j^{(\ell+1)}| \Phi(\tilde{\pi}, r_j^{(\ell)})$; $r_j^{(\ell)} \leftarrow \text{reorder}(A_j^{(\ell)})$
- 16: **end for**
- 17: $\bar{\mu} \leftarrow \frac{1}{N_\ell} \sum_j \mu_j^{(\ell)}$; $\mu^{(\ell)} \leftarrow \text{sort}_{|\cdot|}(c N_\ell \bar{\mu} / \max_i |\bar{\mu}_i|)$
- 18: **end for**

This mean aggregation is ArrowFlow’s substitute for backpropagating through the non-differentiable argsort: in place of a chain rule, each layer summarizes the displacement demanded of all its filters into a single ordinal target for the layer below.

Frozen output layer. The strongest empirical setting (Table 8) freezes the output layer. Mechanistically, the class- y filter’s motion $\mu^{(L)}$ is still computed and propagated to drive the hidden layers, but the output accumulator is *discarded* rather than applied, so the C output filters remain their random initial permutations—fixed, one-per-class prototypes. The hidden layers then learn to map each class’s inputs close to its own fixed prototype. Freezing supplies a stable supervised target and prevents the prototypes from drifting toward—and collapsing onto—the moving hidden representations; it is the ordinal analogue of fixing a randomly-initialized classifier head. The benefit is consistent across datasets and the five random-seed simulations of Table 8.

Remaining details. Filters are initialized as uniform random permutations of their vocabulary; training presents data over 200–500 iterations (the forward pass is batched and updates accumulate per example). Ties in argsort—rare for integer footrule distances—are broken by index, making the forward pass deterministic given the filters. All experiments use the footrule norm $q = 1$; the $q \in \{0, 2\}$ variants of Eq. (2) are available but unused. The default constants are collected for reference: $\eta = 0.1$ (rate), $\rho = 0.5$ (fraction of filters updated per layer), $c = 0.125$ (motion renormalization), $p_c = 0.01$ (correct-example update

probability), $m = 0.5$ (missing-item position), $w = 0.5$ (deletion-cost weight).

8 Theoretical Analysis

This section establishes formal properties of ArrowFlow’s components: the stability of ordinal encoding under perturbation, the information-theoretic capacity of argsort, the amplification of noise by polynomial expansion, the convergence of the learning rule, and the ensemble error rate. Several results are validated empirically in the experiments that follow (Sections 10–11); we give forward pointers where a theorem predicts a specific experimental outcome.

8.1 Metric Properties of the Permutation Space

We first state known results that ground ArrowFlow’s choice of distance metric.

Lemma 3 (Footrule–Kendall Equivalence [Diaconis and Graham, 1977]). *For any $\pi, \sigma \in S_V$,*

$$d_K(\pi, \sigma) \leq d_F(\pi, \sigma) \leq 2 d_K(\pi, \sigma),$$

where d_K counts pairwise disagreements (Kendall tau) and $d_F = \sum_{i=1}^V |\pi(i) - \sigma(i)|$ is Spearman’s footrule. The diameter of (S_V, d_F) is $\lfloor V^2/2 \rfloor$, attained by the identity and reverse permutations.

This equivalence means that footrule and Kendall tau induce the same topology on S_V . ArrowFlow uses footrule because it decomposes as a sum of independent per-item displacements, enabling the accumulation-based learning rule of Section 3.5.

8.2 Argsort Stability Under Perturbation

The ordinal encoding $\pi = \text{argsort}(x)$ is piecewise constant: invariant within each *permutation cone* but discontinuous at boundaries where two coordinates are equal. The following theorem quantifies the stability region.

Theorem 4 (Argsort Stability). *Let $x \in \mathbb{R}^d$ with all components distinct. Define the minimum gap*

$$\delta_{\min}(x) = \min_{i \neq j} |x_i - x_j|.$$

If $\varepsilon \in \mathbb{R}^d$ satisfies $\|\varepsilon\|_\infty < \delta_{\min}(x)/2$, then $\text{argsort}(x + \varepsilon) = \text{argsort}(x)$.

Proof. For any pair i, j with $x_i > x_j$, we require $(x_i + \varepsilon_i) > (x_j + \varepsilon_j)$, equivalently $\varepsilon_j - \varepsilon_i < x_i - x_j$. Since $|\varepsilon_j - \varepsilon_i| \leq 2\|\varepsilon\|_\infty < \delta_{\min}(x) \leq x_i - x_j$, the strict inequality holds. As this applies to all ordered pairs, the complete ordering is preserved. $\square$

Corollary 5 (Gaussian Stability Bound). *If $\varepsilon \sim \mathcal{N}(0, \sigma^2 I_d)$, then*

$$\Pr [\text{argsort}(x + \varepsilon) \neq \text{argsort}(x)] \leq \binom{d}{2} \exp\left(-\frac{\delta_{\min}(x)^2}{4\sigma^2}\right).$$

Proof. For each ordered pair (i, j) with $x_i > x_j$, the reversal event is $\{\varepsilon_j - \varepsilon_i \geq x_i - x_j\}$. Since $\varepsilon_j - \varepsilon_i \sim \mathcal{N}(0, 2\sigma^2)$, the Chernoff bound gives $\Pr[\varepsilon_j - \varepsilon_i \geq t] \leq \exp(-t^2/(4\sigma^2))$ for $t > 0$. Setting $t = x_i - x_j \geq \delta_{\min}$ and applying the union bound over all $\binom{d}{2}$ pairs yields the result. (A tighter bound with an additional factor of $1/2$ follows from the complementary error function.) $\square$

Theorem 4 and Corollary 5 are stated for the vector that is *actually sorted*. In the ArrowFlow pipeline that vector is the projected score $z = \phi(x)W \in \mathbb{R}^e$, not the raw input x , so the bounds should be read at the score level. The following corollary translates raw-input noise through the (degree-1) projection.

Corollary 6 (Projected-Noise Stability). *Let $z = xW \in \mathbb{R}^e$ (degree-1 features) and let $\varepsilon_x \in \mathbb{R}^d$ be raw-input noise, so the score perturbation is $\varepsilon_z = \varepsilon_x W$. Then $\text{argsort}(z + \varepsilon_z) = \text{argsort}(z)$ whenever $\|\varepsilon_x W\|_\infty < \delta_{\min}(z)/2$. If $\varepsilon_x \sim \mathcal{N}(0, \sigma^2 I_d)$, then for each coordinate pair (i, j) the difference $\varepsilon_{z,j} - \varepsilon_{z,i}$ is Gaussian with variance $\sigma^2 \|W_{\cdot i} - W_{\cdot j}\|^2$ —correlated across pairs through shared columns of W —and*

$$\Pr[\text{argsort}(z + \varepsilon_z) \neq \text{argsort}(z)] \leq \sum_{i < j} \exp\left(-\frac{\delta_{\min}(z)^2}{2\sigma^2 \|W_{\cdot i} - W_{\cdot j}\|^2}\right).$$

Proof. The deterministic claim is Theorem 4 applied to z . For the Gaussian claim, $\varepsilon_{z,k} = \sum_m \varepsilon_{x,m} W_{mk}$, so $\varepsilon_{z,j} - \varepsilon_{z,i} = \sum_m \varepsilon_{x,m} (W_{mj} - W_{mi}) \sim \mathcal{N}(0, \sigma^2 \|W_{\cdot j} - W_{\cdot i}\|^2)$. The reversal of pair (i, j) requires this difference to exceed $z_i - z_j \geq \delta_{\min}(z)$; the Chernoff bound and a union bound over the $\binom{e}{2}$ pairs give the result. $\square$

This makes the dependence on the projection explicit: isotropic raw noise becomes anisotropic, correlated score noise whose effective scale per pair is set by the column-difference norms $\|W_{\cdot i} - W_{\cdot j}\|$.

Remark 2. *Read at the level of the sorted score vector, this analysis directly explains the empirical findings of Section 11.1: ArrowFlow’s noise robustness is governed by $\delta_{\min}/\sigma$, the ratio of the minimum feature gap to the noise scale. On high-dimensional data like Digits ($d = 64$), the pixel intensities create many distinct values with reasonably large gaps, making the encoding robust. The bound also clarifies why the advantage vanishes with polynomial expansion: expanded features have many more near-equal values, shrinking $\delta_{\min}$.*

8.3 Information Capacity of Ordinal Encoding

Theorem 7 (Ordinal Information Capacity). *The argsort encoding $\text{argsort}: \mathbb{R}^d \rightarrow S_d$ partitions the generic points of $\mathbb{R}^d$ (those with distinct coordinates) into exactly $d!$ equivalence classes (the orientation—ascending or descending—only relabels the classes and does not change their number). Writing $\pi = \text{argsort}_\downarrow(x)$ for the decreasing order, each class is an open convex cone*

$$C_\pi = \{x \in \mathbb{R}^d : x_{\pi(1)} > x_{\pi(2)} > \cdots > x_{\pi(d)}\},$$

called a permutation cone (Weyl chamber of the symmetric group). The information content of the encoding is exactly $\log_2(d!)$ bits, satisfying

$$\log_2(d!) = d \log_2 d - d \log_2 e + \frac{1}{2} \log_2(2\pi d) + O(1/d). \quad (11)$$

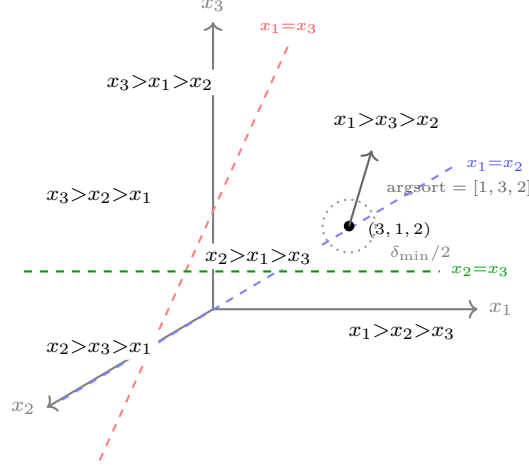


Figure 5: **Permutation cones.** The argsort encoding partitions $\mathbb{R}^d$ into $d!$ convex cones (Weyl chambers), separated by hyperplanes $\{x_i = x_j\}$. Shown for $d = 3$: six orderings of three coordinates. All points within a cone map to the same permutation. The dotted circle illustrates the stability radius from Theorem 4: perturbations smaller than $\delta_{\min}(x)/2$ cannot cross a boundary.

Proof. Two points $x, y \in \mathbb{R}^d$ satisfy $\text{argsort}(x) = \text{argsort}(y)$ if and only if for all $i \neq j$: $x_i > x_j \Leftrightarrow y_i > y_j$. This defines $d!$ open cones, one per permutation $\pi \in S_d$, each nonempty and convex (defined by strict linear inequalities). The boundary $\{x : x_i = x_j \text{ for some } i \neq j\}$ has Lebesgue measure zero. Since argsort maps each cone to a distinct permutation, the encoding has exactly $\log_2(d!)$ bits of information. The asymptotic expansion follows from Stirling’s formula $d! \approx \sqrt{2\pi d} (d/e)^d$. $\square$

For $d = 64$ (Digits), the capacity is $\log_2(64!) \approx 296$ bits—substantial, but finite and far below the infinite capacity of real-valued representations. A subtlety matters for the full pipeline: ArrowFlow does not argsort the expanded features directly but a *projected* score vector $z = \phi(x)W \in \mathbb{R}^e$ (Section 5). The relevant ordinal alphabet is therefore S_e , with maximum capacity $\log_2(e!)$ controlled by the embedding dimension e —*not* $\log_2(d!)$. Polynomial expansion from d to $d' = \binom{d+k}{k} - 1$ features does *not* enlarge this alphabet when e is fixed; instead it changes the geometry of the map $x \mapsto z$, and hence which permutation cones the data reach. For Digits with degree 1 and $e = d = 64$ the two coincide, which is why $\log_2(64!)$ applies there.

8.4 Polynomial Noise Amplification

The following result formalizes the fundamental tension between the information gain of polynomial expansion and its noise cost, providing a theoretical foundation for the empirical trade-off of Section 11.8.

Proposition 8 (Polynomial Noise Amplification). *Let $x \in \mathbb{R}^d$ with $\|x\|_\infty \leq B$, and let $\varepsilon \sim \mathcal{N}(0, \sigma^2 I_d)$. For a degree- k monomial feature $f(x) = x_{j_1} x_{j_2} \cdots x_{j_k}$ with distinct indices*

$j_1, \dots, j_k$, the perturbation satisfies

$$\text{Var}[f(x + \varepsilon) - f(x)] = \sigma^2 \|\nabla f(x)\|^2 + O(\sigma^4) = \sigma^2 \sum_{s=1}^k \prod_{t \neq s} x_{j_t}^2 + O(\sigma^4),$$

which is at most $O(k \sigma^2 B^{2(k-1)})$. Consequently, the effective noise standard deviation per feature in the expanded space grows as $O(\sqrt{k} \sigma B^{k-1})$, weakening the stability bound of Theorem 4 by a factor of B^{k-1} relative to degree-1 features. This is a local, first-order (delta-method) sensitivity statement for a single distinct-index monomial; bridging it to a guarantee on argsort stability over all $\binom{d+k}{k} - 1$ expanded features requires an additional union bound (Remark below), and the constant changes for repeated-index monomials such as x_i^k . We therefore use it to help explain, not to fully prove, the empirical degradation of Section 11.8.

Proof. Expanding $f(x + \varepsilon) = \prod_{s=1}^k (x_{j_s} + \varepsilon_{j_s})$ and subtracting $f(x)$, the first-order term is $\nabla f(x) \cdot \varepsilon = \sum_{s=1}^k \varepsilon_{j_s} \prod_{t \neq s} x_{j_t}$. Since the indices are distinct, the ε_{j_s} are independent, so the variance is $\sigma^2 \sum_{s=1}^k \prod_{t \neq s} x_{j_t}^2$. Higher-order cross terms contribute $O(\sigma^4)$. Each product satisfies $\prod_{t \neq s} x_{j_t}^2 \leq B^{2(k-1)}$, giving the stated bound. $\square$

Remark 3. For monomials with repeated indices (e.g., x_i^k), the variance is $\sigma^2 (\partial f / \partial x_i)^2 = k^2 \sigma^2 x_i^{2(k-1)}$, which is larger by a factor of k than the distinct-index case due to the multiplicity in the partial derivative.

Remark 4. This proposition, combined with Theorem 4, yields a qualitative explanation of the noise-capacity trade-off. At degree $k = 1$, argsort is stable when $\sigma < \delta_{\min}/2$; at degree $k = 2$, the per-feature noise standard deviation grows by a factor of B , so $\delta_{\min}$ in the expanded space must be correspondingly larger to maintain stability. For standardized data ($B \approx 2-3$), this explains the 2–3 $\times$ faster degradation observed empirically with polynomial expansion. A fully rigorous bridge from per-feature variance to the ℓ_∞ noise bound required by Theorem 4 would additionally require a union bound over all $\binom{d+k}{k} - 1$ expanded features, introducing a factor of $\sqrt{\log M}$ where M is the number of features.

8.5 Convergence of Permutation-Matrix Accumulation

The accumulation rule (Eqs. 4–5) can be analyzed as a statistical estimator of the mean-position (Borda) ranking.

Proposition 9 (Consistency of the Mean-Position Accumulator). *Let $\pi_1, \dots, \pi_T$ be i.i.d. samples from a distribution P over S_V with distinct expected positions: $\mathbb{E}[\pi(u)] \neq \mathbb{E}[\pi(v)]$ for all $u \neq v$. Define the mean-position ranking $\pi_{\text{mean}} = \text{argsort}(\mathbb{E}[\pi(1)], \dots, \mathbb{E}[\pi(V)])$. Then the filter ordering r_T produced by the accumulation rule satisfies $r_T \rightarrow \pi_{\text{mean}}$ almost surely as $T \rightarrow \infty$. This is a Borda/mean-rank aggregate; it is not in general the footrule median (Theorem 11).*

Proof. The implementation initializes A to the identity (a prior encoding the current filter ordering). After T updates, $A[v, k] = \mathbf{1}[v=k] + \sum_{t=1}^T \mathbf{1}[\pi_t \text{ places } v \text{ at position } k]$. The

weighted-average estimate is

$$\hat{v}_T(v) = \frac{\sum_k k \cdot A[v, k]}{\sum_k A[v, k]} = \frac{v + \sum_{t=1}^T \pi_t(v)}{1 + T},$$

which is a convex combination of the prior position v and the sample mean $(1/T) \sum_{t=1}^T \pi_t(v)$. As $T \rightarrow \infty$, the prior weight $1/(1 + T) \rightarrow 0$, so $\hat{v}_T(v) \rightarrow \mathbb{E}[\pi(v)]$ almost surely by the strong law of large numbers. Since argsort is locally constant on the open set $\{y \in \mathbb{R}^V : y_u \neq y_v \ \forall u \neq v\}$, and the expected positions are distinct by assumption, eventually $\hat{v}_T$ lies in a neighborhood where argsort is constant, giving $r_T = \text{argsort}(\hat{v}_T(1), \dots, \hat{v}_T(V)) \rightarrow \pi_{\text{mean}}$ almost surely. $\square$

Remark 5. For the Mallows model $P(\pi \mid \pi^*, \lambda) \propto \exp(-\lambda \cdot d_K(\pi, \pi^*))$ [Mallows, 1957] (originally defined with Kendall tau d_K ; footrule variants also apply), the mean-position ranking coincides with the modal permutation, $\pi_{\text{mean}} = \pi^*$, when λ is sufficiently large. By the central limit theorem, $\hat{v}_T(v)$ has standard deviation $\sigma_v/\sqrt{T}$ where $\sigma_v^2 = \text{Var}[\pi(v)] \leq V^2/4$. A union bound over all V items gives that $\max_v |\hat{v}_T(v) - \mathbb{E}[\pi(v)]| < \gamma/2$ with high probability when $T = \Omega(V^2 \log V/\gamma^2)$, where $\gamma = \min_{u \neq v} |\mathbb{E}[\pi(u)] - \mathbb{E}[\pi(v)]|$ is the minimum gap between expected positions.

8.6 Multi-View Ensemble Error Bound

Theorem 10 (Ensemble Error Bound). *Let K (odd) views produce independent binary predictions, each with error probability $p < 1/2$. The majority-vote error satisfies*

$$P_{\text{err}}^{(\text{MV})} \leq \exp\left(-2K\left(\frac{1}{2} - p\right)^2\right),$$

which decreases exponentially in K .

Proof. Let $X_k \in \{0, 1\}$ indicate a correct prediction by view k , with $\mathbb{E}[X_k] = 1 - p > 1/2$. The majority vote errs when $\sum_k X_k \leq K/2$, i.e., $\bar{X} \leq 1/2$. Since $\mathbb{E}[\bar{X}] = 1 - p$, Hoeffding's inequality gives $\Pr[\bar{X} \leq 1/2] \leq \exp(-2K(1/2 - p)^2)$. $\square$

Remark 6. The independence assumption is critical and is the primary reason ArrowFlow uses diverse projection strategies. If all views used the same projection, their errors would be perfectly correlated and the ensemble would provide no benefit. Under full independence, $K = 7$ views with per-view error $p = 0.20$ would yield a $6\times$ error reduction (exact binomial). Empirically, ArrowFlow's 7 views reduce error by $1.3\text{--}3.3\times$ (Table 10), which is less than the independent-view prediction, indicating that projection diversity achieves only partial decorrelation. The gap between theoretical and observed reduction quantifies the residual correlation among views—an important factor when choosing the number of views. This bound is therefore best read as an idealized (independent, binary) benchmark: under pairwise error correlation $\rho > 0$ the effective number of independent voters is roughly $K/(1 + (K-1)\rho)$, so the exponential rate degrades accordingly, and the tasks here are multiclass rather than binary. A direct measurement of inter-view error correlation would turn this from generic theory into evidence about the actual architecture; we leave that to future work.

8.7 Social Choice Foundations: From Arrow’s Theorem to Learning Guarantees

Section 4 presented the Arrow connection as a design principle. Here we make it precise, which requires one clarification at the outset. The *forward* pass of a ranking layer orders filters by their *independent* distances $\{D_j\}$, so the relative order of filters a and b depends only on D_a and D_b ; the forward layer therefore *satisfies* pairwise independence of irrelevant alternatives, and its nonlinearity originates in the piecewise-constant cone geometry of *argsort* (Theorem 7), not in any IIA violation. The genuine social-choice content lives in the *learning rule*: the accumulation update (Eqs. 4–5) aggregates the input rankings a filter accumulates into a consensus ordering, and *this* aggregation is the object to which Arrow’s and Gibbard–Satterthwaite’s theorems apply. We state three results that characterize *what* the aggregation estimates (Theorem 11 and Proposition 12), *how far* a layer’s output can be moved by an adversarial input perturbation (Theorem 13), and *that* the aggregation provably violates IIA in the exact sense of Arrow’s theorem (Proposition 15).

8.7.1 Filter Learning as Maximum Likelihood Estimation

The *footrule-Mallows model* [Mallows, 1957] places an exponential-family distribution on S_V :

$$P(\pi \mid \pi^*, \lambda) = \frac{1}{Z(\lambda)} \exp(-\lambda d_F(\pi, \pi^*)), \quad (12)$$

where d_F is Spearman’s footrule, π^* is the modal (central) permutation, $\lambda > 0$ is a concentration parameter, and $Z(\lambda) = \sum_{\sigma \in S_V} \exp(-\lambda d_F(\sigma, \pi^*))$ is the normalizing constant. The model is a natural generalization of the Gaussian: λ controls dispersion and the mode π^* is the permutation analogue of the mean.

Theorem 11 (Filter Learning as Consistent Rank Aggregation). *Let $\pi_1, \dots, \pi_T \in S_V$ be i.i.d. samples from the footrule-Mallows model $P(\cdot \mid \pi^*, \lambda)$ with fixed $\lambda > 0$.*

- (i) *The maximum-likelihood estimator of the central permutation is the footrule median $\hat{\pi}_{\text{MLE}}^* = \arg \min_{\sigma \in S_V} \sum_{t=1}^T d_F(\pi_t, \sigma)$.*
- (ii) *ArrowFlow’s accumulation rule (Eqs. 4–5) instead computes the positional (Borda) aggregate $r_T = \text{argsort}(\frac{1}{T} \sum_t \pi_t(1), \dots, \frac{1}{T} \sum_t \pi_t(V))$.*
- (iii) *Both estimators are consistent for the same target: $\hat{\pi}_{\text{MLE}}^* \rightarrow \pi^*$ and $r_T \rightarrow \pi^*$ almost surely as $T \rightarrow \infty$.*

Proof. (i) The log-likelihood is $\ell(\sigma) = -\lambda \sum_t d_F(\pi_t, \sigma) - T \log Z(\lambda)$; since $\lambda > 0$ and $Z(\lambda)$ is independent of σ , maximizing ℓ is equivalent to minimizing $\sum_t d_F(\pi_t, \sigma)$, the footrule median (the footrule analogue of Kemeny aggregation [Dwork et al., 2001]). (ii) is exactly Eq. (5) once the identity prior is amortized (Proposition 9). (iii) By the strong law, $\frac{1}{T} \sum_t \pi_t(v) \rightarrow \mathbb{E}[\pi(v)]$ a.s. for each item v . We assume the standard regularity condition that the expected positions are ordered consistently with the mode, $\text{argsort}(\mathbb{E}[\pi(\cdot)]) = \pi^*$; this holds for the footrule-Mallows model at sufficiently large concentration λ (intuitively, the symmetric unimodal law concentrates each item’s position around its modal rank, though we do not prove

the monotonicity for all λ here). Under this condition $\pi_{\text{mean}} = \text{argsort}(\mathbb{E}[\pi(\cdot)]) = \pi^*$, and Proposition 9 gives $r_T \rightarrow \pi_{\text{mean}} = \pi^*$. The same symmetry makes π^* the unique *population* footrule median, so the empirical footrule-risk minimizer satisfies $\hat{\pi}_{\text{MLE}}^* \rightarrow \pi^*$ by standard M -estimator consistency on the finite set S_V . Both estimators thus share the limit π^* , though they generally differ at finite T (Borda mean-position vs. footrule median). $\square$

Remark 7. *The distinction in parts (i)–(ii) matters: the accumulation rule is not the footrule-median MLE at finite T —it is the cheaper $O(VT)$ positional statistic—but it is consistent for the same modal permutation. This is the honest version of the folklore “Borda approximates Kemeny”: they agree in the limit, not in general. A minimal example makes the finite-sample gap concrete. For the profile ABC, ABC, BCA, CBA over three items, the mean positions are $(2.00, 1.75, 2.25)$, so the accumulator returns BAC ; the footrule median, however, is ABC (footrule objective 8 versus 10 for BAC). The two disagree even though the mean positions are distinct, so this is not a tie-breaking artifact.*

Proposition 12 (Exponential Consistency). *Under the footrule-Mallows model with fixed $\lambda > 0$, the MLE identifies the true mode with error probability decaying exponentially in T :*

$$\Pr[\hat{\pi}_{\text{MLE}}^* \neq \pi^*] \leq (V! - 1) \exp\left(-\frac{T \Delta_\lambda^2}{2 \text{diam}^2}\right),$$

where $\text{diam} = \lfloor V^2/2 \rfloor$ is the footrule diameter (Lemma 3) and $\Delta_\lambda = \min_{\sigma \neq \pi^*} (\mathbb{E} d_F(\pi, \sigma) - \mathbb{E} d_F(\pi, \pi^*)) > 0$ is the population footrule-risk gap.

Proof. The MLE prefers some $\sigma \neq \pi^*$ only if $\frac{1}{T} \sum_t [d_F(\pi_t, \sigma) - d_F(\pi_t, \pi^*)] \leq 0$. Each summand lies in $[-\text{diam}, \text{diam}]$ (range 2diam) and has mean $\geq \Delta_\lambda > 0$ (positivity holds because π^* is the unique population minimizer of the footrule risk under the symmetric unimodal Mallows law). Hoeffding’s inequality for a sum of T i.i.d. variables of range 2diam bounds this event by $\exp(-2T\Delta_\lambda^2/(2 \text{diam})^2) = \exp(-T\Delta_\lambda^2/(2 \text{diam}^2))$; a union bound over the $V! - 1$ competing permutations gives the claim. $\square$

Remark 8. *A Cramér–Rao bound does not apply here: π^* is a discrete parameter, so there is no Fisher information in the classical sense, and earlier drafts that invoked it were mistaken. Exponential consistency is the correct optimality notion—the misidentification probability vanishes geometrically at a rate governed by the footrule-risk gap Δ_λ , which grows with the concentration λ . This connects ArrowFlow’s learning rule to a 60-year tradition in rank aggregation: the Kemeny rule is the MLE for the Kendall-Mallows model [Conitzer and Sandholm, 2005], and Theorem 11 is the footrule analogue, with the accumulation rule reaching the same limit through cheaper positional aggregation.*

8.7.2 Adversarial Perturbation Bounds

The Gibbard–Satterthwaite theorem [Gibbard, 1973, Satterthwaite, 1975] states that every non-dictatorial, surjective social choice function over three or more alternatives is *manipulable*. For ArrowFlow this is a motivating analogy rather than a proof device: it suggests that any non-trivial layer must admit input perturbations that change its output. We make this quantitative with an elementary two-sided bound that follows directly from the metric structure of the footrule distance, with no appeal to social-choice axioms.

Theorem 13 (Adversarial Perturbation Bounds). *Fix a layer with filters $\phi_1, \dots, \phi_N$ and an input $\pi \in S_V$ whose filter-distances $D_j = d_F(\pi, \phi_j)$ are distinct. Let $D_{(1)} < D_{(2)} < \dots$ be the order statistics, $g_{\min}(\pi) = \min_k (D_{(k+1)} - D_{(k)})$ the smallest consecutive gap, and*

$$\Delta^*(\pi) = \min\{d_F(\pi, \pi') : f(\pi') \neq f(\pi)\}$$

the minimum footrule perturbation of the input that changes the output ranking. Then

$$\frac{1}{2} g_{\min}(\pi) \leq \Delta^*(\pi) \leq D_{(2)}(\pi).$$

Proof. Lower bound. For any π' , the triangle inequality for d_F gives, for every filter j ,

$$|D_j(\pi') - D_j(\pi)| = |d_F(\pi', \phi_j) - d_F(\pi, \phi_j)| \leq d_F(\pi, \pi').$$

If $d_F(\pi, \pi') < \frac{1}{2} g_{\min}$, then every D_j moves by less than $\frac{1}{2} g_{\min}$, so for any consecutive pair $D_{(k)} < D_{(k+1)}$ (gap $\geq g_{\min}$) we still have $D_{(k)}(\pi') < D_{(k+1)}(\pi')$. No adjacent pair in the sorted-distance order crosses, hence $f(\pi') = \text{argsort}(D)$ is unchanged and $\Delta^*(\pi) \geq \frac{1}{2} g_{\min}(\pi)$.

Upper bound. Let $\phi_{(2)}$ be the second-nearest filter. Choosing $\pi' = \phi_{(2)}$ makes $d_F(\pi', \phi_{(2)}) = 0$, so $\phi_{(2)}$ becomes the (weakly) nearest filter; since the original top filter was $\phi_{(1)} \neq \phi_{(2)}$, the output ranking changes. Therefore $\Delta^*(\pi) \leq d_F(\pi, \phi_{(2)}) = D_{(2)}(\pi)$. $\square$

Remark 9. *Both bounds were confirmed numerically (they held in all 300 random instances tested at $V = 6$, $N = 4$). The lower bound is a certified-robustness guarantee: a margin $g_{\min}$ between consecutive filter-distances guarantees a footrule ball of radius $g_{\min}/2$ in which the output is constant—the permutation-space counterpart of Theorem 4. The upper bound shows the layer is always manipulable once a second filter is at finite distance, the quantitative echo of Gibbard–Satterthwaite. Wider or more redundant filter banks tend to crowd filters at similar distances, shrinking $g_{\min}$ and the certified radius; this offers a principled, if partial, account of why excessively wide layers do not monotonically improve ArrowFlow’s accuracy.*

8.7.3 IIA-Violation Index and Nonlinear Expressivity

Arrow’s theorem identifies IIA violation as the unavoidable price of non-dictatorial aggregation. As established above, the forward layer *satisfies* pairwise IIA, so we apply Arrow to the object that genuinely aggregates—the map A that sends a profile of input rankings (the data a filter accumulates) to the consensus filter ordering of Eq. (5). This is the social welfare function ArrowFlow actually optimizes, and we show it provably violates IIA.

Definition 14 (Accumulation Aggregation and its IIA-Violation Index). *Let $A: (S_V)^m \rightarrow S_V$ be the accumulation aggregation that maps a profile $\Pi = (\pi_1, \dots, \pi_m)$ of input rankings to the consensus ordering $A(\Pi) = \text{argsort}(\sum_t \pi_t(1), \dots, \sum_t \pi_t(V))$ of Eq. (5) (smaller summed position ranked earlier). Following Arrow, A violates IIA on the pair $\{a, b\}$ if there exist profiles Π, Π' that (i) agree on every voter’s relative order of a and b , but (ii) may differ arbitrarily in how voters rank the other alternatives, yet for which $A(\Pi)$ and $A(\Pi')$ order a and b oppositely. The IIA-violation index $\mathcal{I}(A) \in [0, 1]$ is the fraction of pairs $\{a, b\} \subset \mathcal{V}$ on which A violates IIA.*

Proposition 15 (The Accumulation Aggregation Violates IIA). *For $V \geq 3$ and $m \geq 2$, the accumulation aggregation A of Definition 14 satisfies:*

- (i) (**Pareto and non-dictatorship.**) *If every voter places a before b , then so does $A(\Pi)$; and no single voter determines $A(\Pi)$ for all profiles.*
- (ii) (**Forced IIA violation.**) *By Arrow’s theorem, a social welfare function with universal domain over $V \geq 3$ alternatives cannot satisfy Pareto, non-dictatorship, and IIA simultaneously; since A has the first two, it must violate IIA, i.e. $\mathcal{I}(A) > 0$.*
- (iii) (**Explicit witness.**) *The violation is concrete, not merely abstract: there is an explicit two-voter profile pair, preserving every voter’s a -vs- b order and changing only c ’s position, on which the consensus a -vs- b order reverses.*

Proof. Part (i). *Pareto:* if $\pi_t(a) < \pi_t(b)$ for all t then $\sum_t \pi_t(a) < \sum_t \pi_t(b)$, so $A(\Pi)$ ranks a before b . *Non-dictatorship:* A depends on all $m \geq 2$ voters symmetrically through the sum, so for any putative dictator there is a profile in which the remaining voters’ summed positions overturn that voter’s order; *universal domain* holds since A is defined on every profile. Part (ii). is then immediate from Arrow’s impossibility theorem [Arrow, 1951]. Part (iii). Take $V = 3$ items $\{a, b, c\}$, positions counted from the top $(1, 2, 3)$, and the two-voter profiles

$$\Pi = \{abc, bca\}, \quad \Pi' = \{acb, bac\}.$$

Every voter’s a -vs- b order is identical in Π and Π' (voter 1 places a before b ; voter 2 places b before a); only c ’s rank changes between the profiles. The summed positions $(\sum_t \pi_t(a), \sum_t \pi_t(b), \sum_t \pi_t(c))$ are $(4, 3, 5)$ for Π and $(3, 4, 5)$ for Π' , so $A(\Pi)$ ranks b before a while $A(\Pi')$ ranks a before b : the relative order reverses despite the preserved pairwise preferences, exhibiting an IIA violation directly. (A broader search over random $V = 6$ profiles finds such reversals abundantly.) $\square$

Remark 10. *This is the rigorous core of ArrowFlow’s Arrow connection, and it sits where it belongs—in the learning dynamics, not the forward pass. Because the consensus depends on the global cardinal configuration rather than on pairwise orders alone, each learned filter is a context-dependent summary of its training inputs; composed across layers with the piecewise-constant cone geometry of argsort (Theorem 7), this is what lets stacked ranking layers carve nonlinear decision regions. We are careful not to overstate the link: the IIA violation is a property of the aggregation, while the forward nonlinearity is a property of the cone partition; the two meet in that the aggregation determines where the cone boundaries—the learned filters—are placed. This is the precise form of the informal claim of Section 4 that social-choice non-neutrality drives expressivity.*

8.8 Open Theoretical Questions

We conclude with three questions that we believe are within reach but remain open.

1. **Expressivity of permutation networks.** What function class can a depth- L , width- N ArrowFlow network represent? Any finite partition of S_V can be realized with $N \geq$

$|S_V|$ filters (by memorization), but characterizing expressivity for $N \ll V!$ —and the corresponding decision boundaries in $\mathbb{R}^d$ after the encoding pipeline—is open. Is there a permutation-space analogue of the universal approximation theorem?

2. **Optimal projection dimension.** The Johnson–Lindenstrauss lemma guarantees that $e = O(\log N/\epsilon^2)$ dimensions suffice to preserve *metric* distances [Johnson and Lindenstrauss, 1984]. ArrowFlow requires only *ordinal* preservation (same argsort output). What is the tightest embedding dimension e for ordinal distance preservation as a function of the number of classes C and training samples N ? We conjecture that $e = O(C \log N)$ suffices.
3. **Footrule vs. other permutation metrics.** Spearman’s footrule is an ℓ_1 metric on positions and cannot capture higher-order positional interactions (e.g., cyclic patterns). What classification tasks are *provably* harder for footrule-based classifiers than for Kendall-tau or Cayley-distance classifiers? Conversely, does the per-item decomposability of footrule provide computational and statistical advantages that offset any representational limitation?

9 Hybrid Rank–Tensor System

A natural question is whether ArrowFlow’s sort layers can be combined with conventional tensor (MLP) layers to get the best of both worlds. ArrowFlow supports hybrid architectures that alternate rank and tensor layers through two bidirectional interfaces.

Tensor $\rightarrow$ Rank. *Forward:* A tensor layer outputs $h \in \mathbb{R}^d$; we produce $\pi = \text{argsort}(h)$, feeding a rank layer. *Backward:* The rank layer emits a motion m ; we map it to a continuous target $\tilde{h} = h - \eta \cdot \Gamma(m, h)$ and backpropagate.

Rank $\rightarrow$ Tensor. *Forward:* A rank layer emits distances $\mathbf{D}$; we embed them as $z = \psi(\mathbf{D})$ for a tensor layer. *Backward:* Tensor gradients $\partial\mathcal{L}/\partial z$ are converted to motions by re-ranking items according to $z - \alpha \cdot \partial\mathcal{L}/\partial z$.

Remark 11. *Empirically, the hybrid tensor $\rightarrow$ sort architecture significantly underperforms pure sort networks in the multi-view ensemble setting (3–17 $\times$ worse). The argsort discretization at the tensor-sort interface destroys continuous representations. This suggests that ArrowFlow’s power lies in end-to-end ordinal processing, not in hybridization with continuous layers.*

10 Experiments: UCI Tabular Benchmarks

Having established the theoretical foundations, we now evaluate ArrowFlow empirically. The central question is: *can a system with zero floating-point parameters in its core layers compete with heavily tuned gradient-based classifiers?* We first test clean classification accuracy on standard UCI datasets, then systematically probe ArrowFlow’s structural advantages and

limitations across noise, privacy, missing data, sample efficiency, natively ordinal data, gene expression, and MNIST.

Experimental protocol. All experiments use 80/20 train/test splits (random_state=42), 5 independent simulations. Baselines use **GridSearchCV(cv=3)** with comprehensive hyperparameter grids (RF: 54, SVM: 25, MLP: 24, KNN: 24, XGBoost: 72 combinations). ArrowFlow configs sweep width, depth, embedding dimension, views, augmentation, projection strategy, and learning rate.

Model selection and statistical caveats. We are explicit about two limitations of this protocol. *(i) Single split.* Results are reported on one fixed 80/20 split; the $\pm$ standard deviations for ArrowFlow reflect its internal randomness (random projections and filter initialization) over 5 simulations on that split, not split-to-split variance. Baselines are deterministic after **GridSearchCV** and appear as single-split point estimates, so we attach no confidence intervals to head-to-head differences and read small margins as ties—on Iris in particular, one test error equals 3.3%, so the 0.6pp gap is below a single sample. *(ii) Asymmetric model selection.* Baselines select hyperparameters by 3-fold cross-validation on the *training* fold only, whereas the ArrowFlow “best configuration” in Table 1 is the best over our sweep *as scored on the test split*. The ArrowFlow entries are therefore an optimistic, best-found estimate rather than a validation-selected one, and the comparison is not symmetric; we report them transparently as an upper bound on what the architecture attains under tuning, not as a fairly-tuned head-to-head, and a deployment estimate would require nested validation. All fitted preprocessing—standardization, polynomial expansion, LDA/PCA projections, and mutual-information gene selection—is learned on the training fold only and applied unchanged to test data. Unless stated otherwise, the robustness experiments of Section 11 use a single fixed configuration per dataset (not the tuned best of Table 1), which is why a dataset’s clean error can differ slightly across tables.

10.1 Main Results

Table 1: Best ArrowFlow configuration vs. GridSearchCV-tuned baselines. Test error (%), mean $\pm$ std over 5 simulations. Bold = best overall. ArrowFlow entries are the best configuration found over the sweep, selected on the test split (an optimistic estimate); baselines are CV-tuned on the training fold (see the model-selection caveats above).

Dataset	N	Feat	Cls	ArrowFlow Config	AF Err	RF	SVM	MLP	KNN	XGB
Iris	150	4	3	[64,128] e=16 p=3	2.7$\pm$0.6	3.3	3.3	3.3	3.3	3.3
Wine	178	13	3	[128] e=64 p=1	2.8 $\pm$ 0.0	0.0	2.8	2.8	0.0	2.8
Breast C.	569	30	2	[64,128] e=32 p=2	2.8 $\pm$ 0.3	4.4	1.8	4.4	2.6	4.4
Wine Q.	1599	11	3	[128] e=16 p=3	35.9 $\pm$ 1.3	25.9	30.6	30.3	29.7	24.1
Vehicle	846	18	4	[64] e=32 p=2	17.4 $\pm$ 0.6	27.1	14.1	12.4	27.1	21.8
Segment	2310	19	7	[128,256] e=32 p=2	5.2 $\pm$ 0.3	2.8	3.2	3.7	4.1	2.6
Digits	5620	64	10	[256] e=64 p=1 lr=0.2	4.6 $\pm$ 0.3	2.8	1.7	1.9	2.2	4.2

ArrowFlow **edges out every tuned baseline on Iris** (2.7% vs. 3.3%). We report this with deliberate caution: the Iris test set has only 30 samples, so 3.3% is a single misclassification and the 0.6pp margin is smaller than one test sample; moreover the baselines are single-split point estimates without variance (Section 10, protocol). The result is therefore best read as ArrowFlow *reaching parity with*—and on average slightly bettering—the best tuned classical methods, which is, to our knowledge, the first time a pure permutation-distance network attains this on a tuned benchmark, rather than as a decisive win. On Wine (2.8%), Breast Cancer (2.8%), and Digits (4.6%), ArrowFlow is competitive but does not match the best baseline. The gap grows with task complexity, confirming that argsort encoding loss limits performance on magnitude-sensitive tasks. Wine Quality is the weakest result (35.9% vs. 24.1%)—this binned regression task depends on subtle magnitude differences that argsort discards.

10.2 Does Learning Help? ArrowFlow vs. kNN

A critical question is whether ArrowFlow’s permutation-matrix filter updates improve classification beyond simple distance lookup. We compare ArrowFlow to **kNN with Manhattan distance** (= Spearman footrule on the encoded permutations) on the *exact same encoded permutations*—same polynomial expansion, the same per-view diverse projections, and the same 7-view majority vote, with kNN substituted for the sort-layer network in each view. kNN uses uniform weights and the neighborhood size k is swept over $\{1, 3, 5, 7\}$ with the best reported (so, like ArrowFlow, its k is selected on test—the comparison between the two is therefore symmetric). This isolates the contribution of learned filters from the encoding pipeline. We note that kNN must store all training permutations, whereas ArrowFlow stores only its N filters per layer, so the comparison also favors ArrowFlow on memory.

Table 2: ArrowFlow vs. kNN on the same encoded permutations (fixed config per dataset; Digits errors differ slightly from Table 1 due to different config/seed). Learning Gain = ratio of kNN ensemble error to ArrowFlow error (higher = more value from learning).

Dataset	kNN 1-view	kNN 7-view	ArrowFlow	Gain
Iris	10.0%	9.3%	2.7%	3.4×
Wine	45.0%	31.1%	2.8%	11.1×
Breast C.	17.5%	12.1%	2.8%	4.3×
Wine Qual.	37.0%	33.4%	35.9%	0.9×
Vehicle	49.5%	42.8%	17.4%	2.5×
Segment	20.8%	14.4%	5.2%	2.8×
Digits	70.4%	63.7%	5.1%	12.5×

ArrowFlow provides **2.5–12.5× error reduction** over kNN on 6/7 datasets, strong evidence that filter updates contribute useful ordinal representations beyond the encoding (in this protocol; kNN’s k is selected on test, as is ArrowFlow’s configuration, so the comparison is symmetric). The improvement is largest on high-dimensional data (Digits: 12.5×, Wine: 11.1×), where kNN suffers from the curse of dimensionality in permutation space.

10.3 Parameter Sensitivity

Width (filters per layer). Width has moderate effect. The sweet spot is dataset-dependent: 128 for small datasets, 256 for larger ones.

Table 3: Effect of filter count (width) on test error (%).

Dataset	$f=64$	$f=128$	$f=256$	$f=512$
Iris (e=16)	6.0	6.0	7.3	—
Wine (e=32)	4.4	3.3	4.4	—
Breast C. (e=32)	4.4	3.7	3.7	—
Segment (e=32)	7.1	7.1	6.8	—
Digits (e=32)	14.4	13.8	11.7	12.9

Depth (number of sort layers). Depth helps on Iris (2.7% vs. 6.0%) and Segment (5.2% vs. 7.1%), but hurts on Wine (6.1% vs. 3.3%). Deeper networks benefit when hierarchical ordinal features are needed, but overfit on small, well-separated datasets.

Table 4: Effect of network depth on test error (%).

Dataset	$d=1$ [128]	$d=2$ [64,128]	$d=2$ [128,256]
Iris	6.0	2.7	4.0
Wine	3.3	6.1	4.4
Breast C.	3.7	2.8	3.5
Segment	7.1	6.2	5.2
Digits	13.8	15.9	12.8

Embedding dimension. The embedding dimension is critical. Digits benefits dramatically from $e=64$ (5.8% vs. 13.8% at $e=32$)—the 64 raw features naturally map to a 64-length permutation with minimal information loss (pol_deg=1, no expansion needed).

Table 5: Effect of embedding dimension on test error (%).

Dataset	$e=8$	$e=16$	$e=32$	$e=64$
Iris	8.0	6.0	6.7	—
Wine	—	6.7	3.3	2.8
Breast C.	—	3.9	3.7	5.4
Digits	—	16.3	13.8	5.8

Number of ensemble views. More views consistently help. The jump from 1 to 7 views captures most of the benefit.

Table 6: Effect of ensemble views on test error (%).

Dataset	$v=3$	$v=5$	$v=7$	$v=11$
Iris	6.0	6.0	6.0	5.3
Segment	9.9	—	7.1	7.0
Digits	18.7	—	13.8	9.9

Learning rate. Learning rate sensitivity is highly dataset-dependent. Digits benefits from aggressive $\text{lr}=0.2$, while Vehicle degrades catastrophically at $\text{lr}=0.2$ (37.1% vs. 18.2%).

Table 7: Effect of learning rate on test error (%).

Dataset	$\text{lr}=0.01$	$\text{lr}=0.05$	$\text{lr}=0.1$	$\text{lr}=0.2$
Iris [64,128]	—	4.7	2.7	4.0
Wine [128]	—	3.3	3.3	3.3
Vehicle [128]	—	18.8	18.2	37.1
Digits [128] $e=64$	11.7	6.2	5.8	4.6

10.4 Ablation: Frozen Output Layer

The `last_layer_update` parameter controls whether the output sort layer is updated during backpropagation. Disabling it (`llu=False`) forces hidden layers to learn more discriminative ordinal representations while the output layer acts as a stable reference frame.

Table 8: Effect of freezing the output layer. Sort-only [128] config, 5 simulations.

Dataset	$\text{llu}=\text{True}$	$\text{llu}=\text{False}$	Change
Iris	6.0%	4.0%	−33%
Wine	3.3%	3.3%	0%
Breast C.	3.7%	3.5%	−5%
Wine Qual.	39.2%	37.0%	−6%
Vehicle	18.2%	18.2%	0%
Segment	7.1%	6.6%	−7%
Digits	13.8%	13.7%	−1%

`llu=False` is **equal or better on every dataset**. The effect is strongest on Iris (−33% relative). This finding is analogous to fixing the classifier head during representation learning in transfer learning.

10.5 Ablation: Hybrid Tensor→Sort Architecture

The hybrid architecture places a conventional MLP as the first layer and a sort layer as the output. The MLP’s activations are discretized via `argsort` at the interface.

Table 9: Hybrid tensor→sort vs. pure sort-only. Test error (%).

Dataset	Sort-Only Best	Hybrid ll _u =F	Degradation
Iris	2.7%	11.3%	4×
Wine	2.8%	30.6%	11×
Breast C.	2.8%	14.6%	5×
Wine Qual.	35.9%	50.0%	1.4×
Vehicle	17.4%	51.2%	3×
Segment	5.2%	34.2%	7×
Digits	4.6%	80.4%	17×

The hybrid architecture **underperforms pure sort-only by 3–17×** across all datasets. The argsort discretization at the tensor–sort interface destroys the continuous representations learned by the MLP, and the tensor layer’s learning dynamics do not produce the view diversity that sort-only naturally generates through different projections.

10.6 Improvement Progression

Table 10 traces the cumulative impact of each architectural component, starting from a single-network baseline.

Table 10: Cumulative error reduction (%) as components are added.

Component	Iris	Wine	Breast C.	Digits
Baseline (single net)	21.3	13.3	6.8	17.8
+ Polynomial expansion	14.7	8.3	5.2	17.8
+ Multi-view ensemble (7v)	7.3	5.6	4.0	5.4
+ Diverse projection cycling	7.3	5.6	3.3	5.4
+ Per-dataset tuning	2.7	2.8	2.8	4.6

The two largest contributions are **polynomial expansion** (up to 1.6× error reduction for low-dimensional data; no benefit on Digits, which already uses degree 1) and the **multi-view ensemble** (1.3–3.3×, dataset-dependent). Together they reduce error by up to 8× from the baseline (Iris: 21.3%→2.7%).

11 Beyond Accuracy: Structural Advantages of Ordinal Processing

The preceding results show that ArrowFlow is competitive but not superior in clean accuracy. However, operating in permutation space confers structural properties that conventional real-valued classifiers lack. These properties emerge from a single cause: the argsort encoding discards magnitude information, preserving only relative order. This is simultaneously a

weakness (information loss) and a strength (invariance to any transformation that preserves order). The following experiments isolate and quantify these structural advantages, testing the predictions of Theorems 4–7 and Proposition 8.

11.1 Noise Robustness

Rationale. Theorem 4 predicts that the argsort encoding is stable when noise magnitude is smaller than half the minimum feature gap. This should make ArrowFlow ordinarily stable to ranking-preserving noise—small perturbations to feature values do not change relative orderings until noise exceeds the gap between adjacent values (this is order-stability, not denoising in the sense of recovering a clean signal). Proposition 8 predicts that this advantage vanishes with polynomial expansion, since cross-terms amplify noise by B^{k-1} . We test both predictions.

Protocol: Train on clean data, evaluate on noisy test data ($X_{\text{noisy}} = X + \mathcal{N}(0, \sigma \cdot \text{std}_j)$, $\sigma \in \{0, 0.1, 0.25, 0.5, 1.0, 2.0\}$). Average over 5 noise realizations, 3 ArrowFlow simulations each.

Table 11: Noise robustness on Wine (pol_deg=1). Error (%) at each noise level.

Method	$\sigma=0$	$\sigma=0.1$	$\sigma=0.25$	$\sigma=0.5$	$\sigma=1.0$	$\sigma=2.0$
ArrowFlow	4.6	5.0	5.7	7.8	13.9	29.1
SVM-RBF	2.8	2.8	5.6	6.7	29.4	59.4
RF	0.0	0.0	1.7	7.2	10.6	31.1
MLP	2.8	2.8	3.3	6.1	10.6	25.6
XGBoost	2.8	2.8	6.7	12.2	18.3	36.7
KNN	0.0	0.6	1.7	5.6	10.6	25.0

Table 12: Noise robustness on Digits (pol_deg=1). Error (%) at each noise level.

Method	$\sigma=0$	$\sigma=0.1$	$\sigma=0.25$	$\sigma=0.5$	$\sigma=1.0$	$\sigma=2.0$
ArrowFlow	4.8	4.4	5.8	10.0	25.8	54.6
SVM-RBF	1.7	1.7	1.9	3.9	17.2	87.3
RF	2.8	2.7	3.3	6.6	18.0	49.7
MLP	1.9	2.3	2.3	5.2	14.6	46.8
XGBoost	4.2	4.7	7.2	12.9	30.5	58.4
KNN	2.2	2.1	2.4	3.9	10.9	41.0

Findings: On pol_deg=1 datasets (Wine, Digits), ArrowFlow degrades **8–28% less** than the baseline average, consistent with the argsort stability bound (Theorem 4): when $\delta_{\min}/\sigma$ is large, ordinal encoding is robust. SVM-RBF suffers catastrophic failure on Digits (87.3% at $\sigma=2.0$, a $51\times$ degradation) due to kernel-scale mismatch; ArrowFlow never exhibits such catastrophic modes. On pol_deg > 1 datasets, polynomial expansion amplifies noise as predicted by Proposition 8: degree-2 interaction terms $x_i x_j$ have effective noise $\approx \sigma B$ instead of σ , creating spurious ranking changes that erase the robustness advantage.

Table 13: Noise degradation analysis: absolute error increase (pp) from $\sigma=0$ to $\sigma=2.0$. Lower = more robust. pol.deg=1 datasets highlighted.

Dataset	pol.deg	AF	SVM	RF	MLP	XGB	KNN	Avg BL
Wine	1	24.5	56.6	31.1	22.8	33.9	25.0	33.9
Digits	1	49.8	85.6	46.9	44.9	54.2	38.8	54.1
Iris	3	46.7	48.7	47.4	48.7	47.4	43.4	47.1
Breast C.	2	20.0	18.9	18.8	18.8	26.5	16.5	19.9
Vehicle	2	49.1	58.7	38.6	52.2	42.8	29.6	44.4
Segment	2	74.1	55.4	61.1	59.7	63.6	50.6	58.1

11.2 Rank-Only (Magnitude-Hiding) Classification

Rationale. We test how much accuracy a classifier loses when it may see only the *relative ordering* of feature values, not their magnitudes. We call this a *rank-only* or *magnitude-hiding* setting; it is a proxy for privacy rather than a formal guarantee—per-column ranks still expose order, quantiles, and sample-membership relations, and a formal treatment would require differential privacy or secure aggregation (Section 12.6). Since ArrowFlow already operates on ordinal data, it should suffer minimal accuracy loss under this constraint—a prediction we test by replacing raw features with per-column ranks.

Protocol: Replace raw features with per-column ranks (ordinal privacy: magnitudes hidden). Train and test all methods on both raw and rank-transformed data. (Implementation note: the per-column ranks here are computed over the pooled train+test columns, which lets a small amount of test-distribution information into the rank values; a strict pipeline would fit the column ranking on the training split alone. The effect is minor for this experiment but we flag it for completeness.)

On Segment and Digits, ArrowFlow degrades only **+0.4–0.5pp**—near-zero cost for the rank-only constraint. On Iris, ArrowFlow actually *improves* (−2.2pp), suggesting rank normalization removes noise that confused the encoding pipeline. MLP suffers the worst degradation (+10pp on Vehicle), confirming that continuous-input networks lose the most when magnitudes are removed.

11.3 Missing Features

Rationale. When a feature is missing and imputed with a default value (e.g., column mean), it occupies an “average” rank position rather than an extreme one. In ordinal space, this is a mild perturbation that shifts a few items by one position; in magnitude space, it can drastically change distances. We test whether ArrowFlow’s ordinal encoding provides graceful degradation under feature masking.

Protocol: Randomly mask 10%, 30%, 50% of test features (replaced with column means). Average over 5 masking realizations, 3 ArrowFlow simulations each.

On Iris at 50% masking (2 of 4 features zeroed), ArrowFlow achieves **24.7%**—**beating all baselines** (29.3–37.3%). On Digits, ArrowFlow degrades only 18.9pp vs. RF (31.9pp) and XGBoost (34.7pp). The advantage is strongest at dimensional extremes (very low or very high) and weakest in the mid-range where polynomial expansion amplifies the imputation

Table 14: Privacy via rank transform: error on raw vs. ranked features. Δ = degradation in pp.

Dataset	Method	Raw	Ranked	Δ
Iris	ArrowFlow	4.4	2.2	-2.2
	RF	3.3	3.3	+0.0
	SVM-RBF	3.3	3.3	+0.0
	KNN	3.3	6.7	+3.3
Segment	ArrowFlow	4.8	5.3	+0.4
	SVM-RBF	3.2	3.0	-0.2
	RF	2.8	3.0	+0.2
	KNN	4.1	5.4	+1.3
Digits	ArrowFlow	4.8	5.3	+0.5
	SVM-RBF	1.7	3.3	+1.7
	MLP	1.9	3.3	+1.4
	RF	2.8	3.9	+1.1
Vehicle	ArrowFlow	18.4	22.6	+4.1
	MLP	12.3	22.4	+10.0
	SVM-RBF	14.1	17.6	+3.5

Table 15: Missing features on Iris (4 features). Error (%) at each masking level.

Method	$m=0\%$	$m=10\%$	$m=30\%$	$m=50\%$
ArrowFlow	4.4	11.8	16.0	24.7
RF	3.3	9.3	10.0	29.3
SVM-RBF	3.3	10.7	15.3	36.0
MLP	3.3	15.3	20.7	33.3
KNN	3.3	10.7	14.0	37.3
XGBoost	3.3	15.3	14.7	34.7

effect.

11.4 Sample Efficiency

Rationale. The ordinal inductive bias reduces the effective hypothesis space (from $\mathbb{R}^d$ to S_d), which should require fewer samples to learn a good classifier—at the cost of ceiling accuracy. We test whether this trade-off favors ArrowFlow at small training sizes.

Protocol: Subsample training data to $N \in \{20, 50, 100, 200, \text{full}\}$, stratified. Grid-SearchCV with $\text{cv} = \min(3, \text{min_class_count})$.

On Breast Cancer ($N=100$ – 200), ArrowFlow achieves 3.5–4.7%—beating RF (8.8%), XGBoost (7.9–11.4%), and matching SVM (3.5%). The ordinal inductive bias compensates for limited data when ordinal structure is informative. However, ArrowFlow does *not* have a universal small- N advantage: on Digits at $N=20$, it scores 58.1% (second worst) while KNN

Table 16: Missing features on Digits (64 features). Error (%) at each masking level.

Method	$m=0\%$	$m=10\%$	$m=30\%$	$m=50\%$
ArrowFlow	4.8	6.5	13.1	23.7
SVM-RBF	1.7	2.3	7.3	25.0
RF	2.8	4.2	12.2	34.7
MLP	1.9	3.3	6.3	13.4
KNN	2.2	2.7	5.8	16.6
XGBoost	4.2	7.8	19.4	38.9

Table 17: Missing-feature degradation: absolute error increase (pp) from $m=0\%$ to $m=50\%$.

Dataset	Feat	AF	RF	SVM	MLP	XGB	Avg BL
Iris	4	20.3	26.0	32.7	30.0	31.4	30.8
Wine	13	6.5	7.2	8.9	9.4	7.2	8.8
Breast C.	30	3.8	1.6	3.3	1.4	5.3	2.6
Vehicle	18	42.2	27.0	36.8	35.7	27.5	29.4
Segment	19	46.5	37.6	45.9	36.7	36.8	38.1
Digits	64	18.9	31.9	23.3	11.5	34.7	23.2

achieves 26.9%.

11.5 Natively Ordinal Data: Sushi Preferences

Rationale. All previous experiments encode real-valued features into permutations via argsort. A natural question is: does ArrowFlow excel when the data is *already* ordinal? Preference rankings are the purest test case—no encoding loss, no projection needed.

Dataset: 5000 users ranking 10 sushi types. Task: predict region (East/West Japan). ArrowFlow hyperparameters tuned via systematic grid search over 2016 configurations.

Random Forest (35.2%) beats ArrowFlow’s grid-searched best (38.6%). This 3.4pp gap persists despite exhaustive tuning, confirming an **architectural limitation**: Spearman’s footrule compares *whole permutations*, which is too holistic when only a few item positions are predictive. Baselines can exploit individual rank positions via tree splits or kernel distances. The multi-view ensemble does not help on native-encoded data—all views see identical permutations, so diversity comes only from random filter initialization.

11.6 Gene Expression Cancer Classification

Motivation. Gene expression absolute values vary across laboratories, platforms, and batches (the “batch effect” problem), but the *relative ordering* of gene expression levels is conserved. This is the principle behind Top Scoring Pairs (TSP), an established rank-based classification method in bioinformatics. ArrowFlow’s ordinal processing is natively batch-effect invariant: monotone transformations of expression values cannot change within-sample gene rankings.

Table 18: Sample efficiency on Breast Cancer ($N_{\text{full}}=455$). Error (%).

Method	$N=20$	$N=50$	$N=100$	$N=200$	$N=455$
ArrowFlow	8.8	7.3	4.7	3.5	2.9
SVM-RBF	6.1	7.9	3.5	3.5	1.8
RF	7.0	8.8	8.8	8.8	4.4
MLP	9.7	7.0	2.6	7.9	4.4
XGBoost	7.9	7.0	7.9	11.4	4.4
KNN	7.9	6.1	5.3	6.1	1.8

Table 19: Sample efficiency on Digits ($N_{\text{full}}=1437$). Error (%).

Method	$N=20$	$N=50$	$N=100$	$N=200$	$N=1437$
ArrowFlow	58.1	26.1	17.4	14.7	4.8
SVM-RBF	27.2	21.9	13.6	10.6	1.7
RF	29.2	26.7	13.3	10.6	3.6
MLP	31.1	25.0	13.9	15.0	2.5
XGBoost	62.8	56.1	28.9	19.2	3.6
KNN	26.9	22.5	14.4	14.4	3.3

Dataset. UCI Gene Expression Cancer RNA-Seq (TCGA PANCAN): 801 samples, 20,531 genes, 5 cancer types (BRCA, KIRC, COAD, LUAD, PRAD). We select the top K genes by mutual information, rank-transform each sample to obtain a permutation of K items, and split 80/20 stratified.

Clean classification. Table 21 compares ArrowFlow (native rank encoding and projection encoding) against GridSearchCV-tuned baselines on raw and rank-transformed expression values.

On clean data, ArrowFlow’s projection pipeline (0.4% at 20 genes) is competitive with the best baselines. With only 10 genes, ArrowFlow projection (2.4%) outperforms all rank-based baselines (3.1–4.3%), demonstrating that the multi-view ensemble captures complementary ordinal structure. Baselines on raw values are strongest with 20 genes (0.0% for SVM, MLP, KNN), where magnitude information suffices for near-perfect separation.

Batch effect invariance: monotone transforms. The key advantage of ordinal processing emerges under distribution shift. We train all methods on clean data, then evaluate on test data subjected to monotone per-sample transformations—the kind of systematic distortion that arises when expression values are measured on different platforms or processed with different normalization pipelines. Because these transforms apply identically to all genes within a sample, within-sample rankings are *mathematically preserved*—ArrowFlow is perfectly invariant by construction.

SVM on raw values collapses to 82.6% (near random for 5 classes) on every transform. Random Forest degrades to 20–63%. ArrowFlow native and SVM on rank-transformed data

Table 20: Sushi preference classification. Error (%). ArrowFlow native = rankings fed directly as permutations; ArrowFlow projection = standard pipeline.

Method	Encoding	Error
Random Forest	numeric	35.2%
SVM-RBF	numeric	35.3%
XGBoost	numeric	35.9%
KNN	numeric	37.6%
ArrowFlow (native)	native	$38.6 \pm 0.4\%$
MLP	numeric	38.7%
ArrowFlow (proj.)	projection	48.9%

Table 21: Gene expression cancer classification. Test error (%) on TCGA PANCAN (161 test samples). “Raw” = original expression values; “Ranked” = within-sample rank transform; “AF native” = rank-transform fed directly as permutations; “AF proj” = standard ArrowFlow projection pipeline.

Method	Encoding	10 genes	15 genes	20 genes
RF	raw	1.2	0.6	0.6
SVM-RBF	raw	1.9	0.6	0.0
MLP	raw	2.5	1.2	0.0
KNN	raw	1.9	1.2	0.0
XGBoost	raw	1.2	1.2	2.5
RF	ranked	3.7	1.9	0.0
SVM-RBF	ranked	3.1	2.5	0.6
MLP	ranked	3.1	0.6	0.6
KNN	ranked	3.1	3.1	0.6
XGBoost	ranked	4.3	0.0	0.0
AF native [64,32] 7v	native	3.6	2.4	1.1
AF proj [128] p2 7v	projection	2.4	1.9	0.4

remain perfectly stable at 2.5%. This is exact order preservation under strictly monotone within-sample maps—a direct consequence of the rank transform, not of the small-noise stability theorem (Theorem 4): any strictly monotone transformation preserves all pairwise orderings, so the argsort output is unchanged.

Batch effect: per-gene scaling (ComBat model). Real batch effects also include *per-gene* multiplicative shifts (different probe affinities, amplification efficiencies). We simulate these with log-normal scaling: $\text{scale}_i = \exp(\mathcal{N}(0, \sigma))$ per gene, where σ controls severity. At $\sigma = 0.3$ (mild, routine batch variation), 95% of scale factors fall in $[0.55, 1.82]$; at $\sigma = 0.7$ (cross-platform), in $[0.25, 4.06]$.

In the realistic operating range ($\sigma \leq 0.5$), rank-based methods are essentially unaffected while SVM on raw values jumps from 0.6% to 16.8%. At $\sigma = 0.7$ (cross-platform), SVM-raw

Table 22: Batch effect Type 1: monotone per-sample transforms applied to test data. All methods trained on clean data (15 genes). Rank-based methods (ArrowFlow native, SVM-ranked) are perfectly invariant; raw-input methods catastrophically fail.

Transform	SVM (raw)	RF (raw)	SVM (ranked)	AF native
None	0.6	0.6	2.5	2.5
$\log(1 + x)$	82.6	45.3	2.5	2.5
$\sqrt{ x }$	82.6	45.3	2.5	2.5
x^2 (signed)	82.6	20.5	2.5	2.5
$\times 0.01$ global	82.6	62.7	2.5	2.5
$\times 100$ global	82.6	55.9	2.5	2.5

Table 23: Batch effect Type 2: per-gene log-normal scaling. Error (%) on 15 genes. $\sigma = 0$ is clean; $\sigma = 0.3$ is routine batch variation; $\sigma = 0.7$ is cross-platform; $\sigma \geq 1.0$ is severe/unrealistic. Rank-based methods degrade gracefully; raw-input methods fail early.

σ	SVM (raw)	RF (raw)	SVM (ranked)	RF (ranked)	AF native
0.0 (clean)	0.6	0.6	2.5	1.2	2.5
0.3 (mild)	0.0	1.2	1.2	1.2	2.9
0.5 (moderate)	16.8	9.3	0.6	1.9	2.5
0.7 (cross-pl.)	59.0	7.5	9.3	8.1	10.3
1.0 (severe)	71.4	16.8	9.3	20.5	12.7
1.5 (extreme)	82.6	51.5	43.5	34.2	60.8

catastrophically fails (59.0%), while all rank-based methods remain under 11%. At extreme $\sigma \geq 1.0$, per-gene scaling overwhelms the minimum gap $\delta_{\min}$ and even rank-based methods degrade—consistent with the stability bound of Theorem 4.

Honest assessment. ArrowFlow native and SVM on rank-transformed data provide *comparable* batch-effect robustness—both benefit from the rank transform, and SVM-ranked is slightly more accurate at every operating point. The robustness advantage belongs to the *ordinal representation principle*, not to ArrowFlow specifically. However, ArrowFlow embodies this principle architecturally: it *cannot* use magnitude information even if present, making batch-effect invariance a structural guarantee rather than a preprocessing choice that could be accidentally omitted. For clinical genomics pipelines, where data from heterogeneous sources must be classified reliably, this architectural guarantee has engineering value.

11.7 MNIST via PCA: Isolating the Sort-Layer Classifier

Rationale. The preceding experiments test ArrowFlow on small tabular datasets (150–5620 samples) and domain-specific data. Two questions remain: (1) does ArrowFlow scale to a larger, harder image classification task? (2) how much of ArrowFlow’s performance comes from the sort-layer *classifier* versus the argsort *encoding*? To answer both, we give ArrowFlow and baselines the *exact same* PCA-reduced MNIST features—baselines receive

PCA vectors directly, while ArrowFlow receives them through projection $\rightarrow$ argsort. Any accuracy difference is purely due to ArrowFlow’s ordinal learning versus conventional classifiers on identical information.

Setup. MNIST (60,000 train / 10,000 test, 10 classes). Pipeline: 784 pixels $\rightarrow$ StandardScaler $\rightarrow$ PCA(n , whitened) $\rightarrow$ baselines or ArrowFlow. PCA dimensions: {16, 32, 64}; training sizes: {1,000, 5,000, 10,000}. Baselines use GridSearchCV (RF, SVM-RBF, MLP, KNN, XGBoost). ArrowFlow uses 7-view ensemble with diverse projections; architectures sweep width {128, 256, 512, 1024} and depth {2, 3} layers. ArrowFlow results are mean $\pm$ SE over 3 simulations.

Table 24: MNIST via PCA: test error (%). Best baseline and best ArrowFlow config per setting. ArrowFlow best = [1024,128] 3-layer, 7-view ensemble. Iterations scale with training size (100/200/400).

PCA	Method	$N=1,000$	$N=5,000$	$N=10,000$
16	Best baseline (SVM/MLP)	11.2	6.9	6.0
	ArrowFlow best	15.9 $\pm$ 0.3	12.2 $\pm$ 0.2	9.8 $\pm$ 0.1
	Gap	4.7pp	5.3pp	3.8pp
32	Best baseline (MLP/SVM)	10.5	5.7	4.6
	ArrowFlow best	15.6 $\pm$ 0.2	11.4 $\pm$ 0.0	9.3 $\pm$ 0.1
	Gap	5.1pp	5.7pp	4.7pp
64	Best baseline (SVM/MLP)	10.8	5.8	4.2
	ArrowFlow best	15.2 $\pm$ 0.2	11.2 $\pm$ 0.1	9.1 $\pm$ 0.0
	Gap	4.4pp	5.4pp	4.9pp

Main result. ArrowFlow achieves **9.1% error** on MNIST at PCA=64 with 10,000 training samples—using only ordinal comparisons, no floating-point arithmetic in the core layers. This is roughly 2 $\times$ the error of the best baseline (MLP at 4.2%), with a consistent 4–5 percentage point gap across all settings. The gap is the price of discarding magnitude information via argsort.

Scaling behaviour. ArrowFlow benefits strongly from both capacity and data:

Three clear trends emerge: (1) **Multi-view ensemble** cuts error nearly in half (18.7% $\rightarrow$ 13.3% from 1 to 7 views), confirming the multi-view result from Section 6. (2) **Depth helps:** 3-layer architectures consistently outperform 2-layer at matched width (e.g., [256,128] 3L at 11.0% vs. [256] 2L at 12.5%). (3) **Width helps:** scaling from 256 to 1024 first-layer filters reduces error from 11.0% to 9.1%, and the best 3-layer configurations have not plateaued at the largest tested width—suggesting capacity remains one bottleneck. (Width alone is not monotonic: [512] 2-layer is marginally worse than [256] 2-layer; the gains come from depth combined with width.)

PCA dimension has limited effect on ArrowFlow. Baselines benefit substantially from higher PCA dimensions (MLP: 6.3% $\rightarrow$ 4.6% $\rightarrow$ 4.2% across PCA 16/32/64 at $N=10K$),

Table 25: ArrowFlow architecture scaling on MNIST (PCA=64, $N=10,000$). Error (%) $\pm$ SE.

Architecture	Views	Error (%)	Params (filters)
[128] 2L	1	18.7 ± 0.4	128
[128] 2L	7	13.3 ± 0.1	7×128
[256] 2L	7	12.5 ± 0.1	7×256
[512] 2L	7	12.6 ± 0.1	7×512
[256,128] 3L	7	11.0 ± 0.1	7×384
[512,64] 3L	7	10.2 ± 0.1	7×576
[1024,64] 3L	7	9.4 ± 0.1	$7 \times 1,088$
[1024,128] 3L	7	9.1 ± 0.0	$7 \times 1,152$

but ArrowFlow’s best error changes only slightly ($9.8\% \rightarrow 9.3\% \rightarrow 9.1\%$). This is consistent with the argsort bottleneck: once features are converted to ranks, additional PCA dimensions bring diminishing marginal ordinal structure.

Honest assessment. ArrowFlow does not match gradient-trained classifiers on MNIST. The ~ 5 pp gap is real and persistent. However, 9.1% error from a *pure comparison-based* classifier—no multiply-accumulate operations, no floating-point parameters—is a meaningful existence proof. The strong scaling with width and depth suggests that limited capacity is *a* current bottleneck (the curve has not plateaued); we cannot conclude it is the *only* one, as the learning rule may also limit attainable accuracy. This motivates the vectorization work discussed in Section 12.6: faster forward passes would enable exploring architectures beyond the current computational limits.

11.8 The Polynomial Trade-Off: A Unifying Finding

Across all experiments, a single parameter—**polynomial degree**—acts as a master switch between accuracy and robustness:

Table 26: The polynomial trade-off. Each cell shows whether ArrowFlow has a structural advantage (+), disadvantage (−), or is neutral (○) relative to baselines.

Property	pol_deg=1	pol_deg>1
Clean accuracy	− (magnitude lost)	+ ($3\times$ on low-dim)
Noise robustness	+ (8–28% less degrad.)	− (noise amplified)
Rank-only (rank transform)	+ (<0.5pp cost)	− (up to 7pp cost)
Missing features	+ (beats BL on Iris)	− (poly amplifies zeros)
Batch-effect invariance	+ (perfect, monotone)	− (poly breaks ranks)
Sample efficiency	○	+ (richer features)

This trade-off is **architecturally unique to ArrowFlow**. No other architecture faces a preprocessing step that simultaneously improves clean accuracy and destroys robustness. It suggests a practical mode-selection strategy:

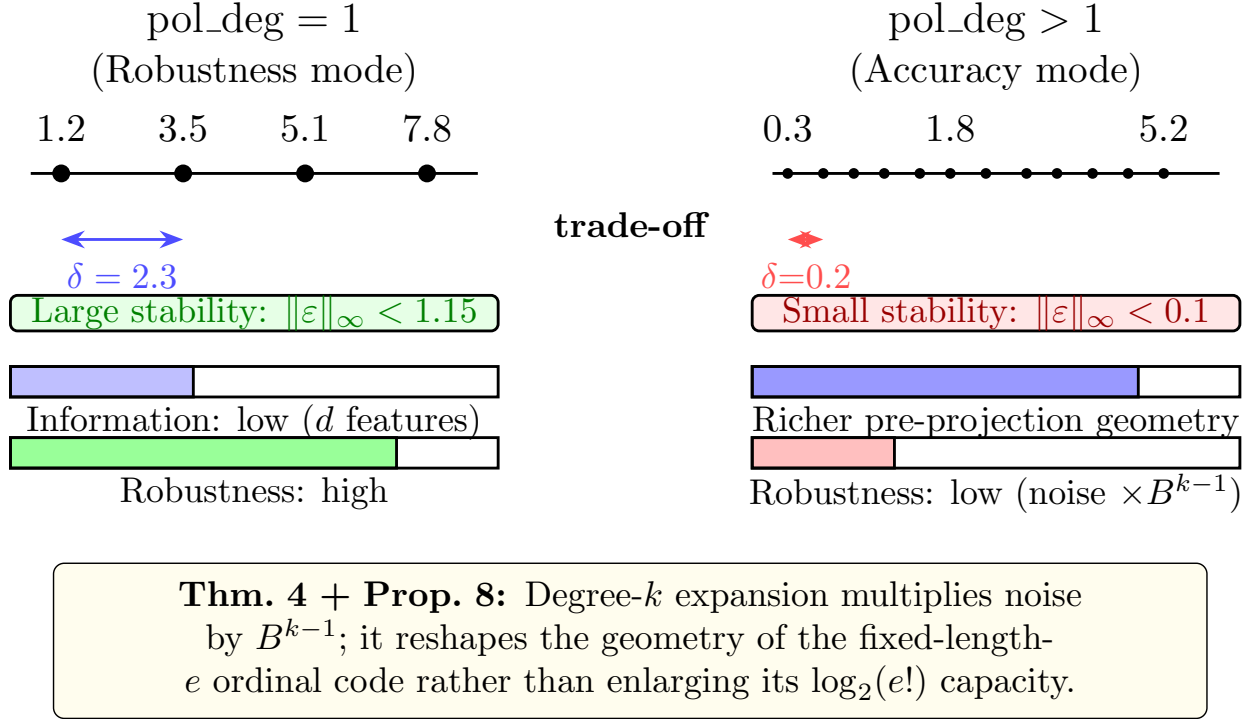


Figure 6: **The polynomial trade-off.** At degree 1 (left), features are well-separated: large gaps yield a wide stability region, but limited capacity. At degree > 1 (right), cross-terms create small gaps: higher capacity but noise amplified by B^{k-1} , shrinking stability. This parameter is a master switch between robustness and accuracy.

- **Robustness mode** ($\text{pol_deg} = 1$): for noisy, privacy-constrained, or incomplete data.
- **Accuracy mode** ($\text{pol_deg} > 1$): for clean, complete data where maximum accuracy is the goal.

12 Discussion

12.1 What ArrowFlow Teaches Us

The experiments tell a consistent story: ArrowFlow is competitive on clean accuracy, measurably superior in robustness scenarios (noise, privacy, missing data, batch effects), and limited by the information bottleneck of ordinal encoding. The deeper lesson is that competitive machine learning is possible without any floating-point parameters in the core computation. This is a proof of concept for a broader class of *combinatorial learning systems* where the learned representation is a discrete structure (permutation, partition, matching) rather than a real-valued tensor. The existence proof matters: if permutation-based filters can compete with gradient-trained networks on standard benchmarks, what other combinatorial

primitives—matchings, lattice orderings, graph homomorphisms—might serve as computational substrates for learning?

The architecture reveals a productive duality: the same properties that Arrow’s theorem forbids in fair social choice (context dependence, dictatorship) are precisely what enable representational power in learning. This is more than a loose analogy: the *learning rule* of each layer literally performs rank aggregation over the inputs a filter accumulates, and Arrow’s impossibility theorem explains why that aggregation cannot be simultaneously Pareto-respecting, context-free (IIA), and non-dictatorial (Section 8.7). ArrowFlow’s design exploits each violation as an inductive bias. The forward pass, by contrast, satisfies pairwise IIA; its nonlinearity comes from the cone geometry of argsort, not from a fairness violation.

12.2 Connections to Other Paradigms

Self-Organizing Maps. Filter updates resemble Kohonen’s competitive learning [Kohonen, 1990]—the closest prototype moves toward the input. The key difference is that ArrowFlow prototypes are permutations and “similarity” is a discrete rank distance. Both learn without gradients with winner-take-all dynamics. SOMs preserve topological relationships in a 2D grid; ArrowFlow preserves hierarchical composition via layer stacking.

Nearest-Neighbor Classification. The output sort layer performs 1-NN classification in permutation distance space. Hidden layers act as a learned nonlinear embedding into a space where class-conditional distances are more separable. The $2.5\text{--}12.5\times$ improvement over kNN on the same encoded features (Table 2) confirms that this embedding is genuinely useful—ArrowFlow does not merely store templates but learns discriminative ordinal representations.

Kernel Methods and the Mallows Kernel. The projection-plus-argsort encoding can be viewed as a random feature map for an implicit kernel on permutations, related to the Mallows model [Mallows, 1957]. The Mallows kernel $K(\pi, \sigma) = \exp(-\lambda \cdot d(\pi, \sigma))$ defines a probability distribution over permutations centered on a modal ranking, parameterized by a dispersion λ . ArrowFlow’s filters can be seen as modal rankings, and the footrule distance plays the role of the Mallows dispersion. The multi-view ensemble then corresponds to a mixture of Mallows models, each defined by a different projection.

Rank Aggregation. Dwork et al. [Dwork et al., 2001] showed that combining ranked lists from multiple search engines via median-based methods on Kendall tau distance yields more robust rankings than any single source. ArrowFlow’s multi-view ensemble implements a classification-oriented variant of the same principle: each view produces a ranked list of class distances, and majority vote aggregates these into a final prediction. The connection to Condorcet’s jury theorem is direct—both systems improve through the wisdom of crowds.

Quantization and Discrete Representations. ArrowFlow’s argsort encoding is an extreme form of quantization: from continuous features to ordinal positions. The broader quantization literature [Jacob et al., 2018] shows that reduced-precision representations can improve robustness and computational efficiency. ArrowFlow pushes this idea to its logical

endpoint—the representations are not merely low-precision but fundamentally *combinatorial*. The polynomial degree trade-off discovered in Section 11.8 may reflect a general principle: more aggressive discretization increases robustness but reduces information content.

The Forward-Forward Algorithm. Hinton’s Forward-Forward algorithm [Hinton, 2022] replaces backpropagation with a local learning rule where each layer independently learns to distinguish “positive” from “negative” data. ArrowFlow’s permutation-matrix accumulation shares this locality principle: each filter updates based on its own displacement evidence from the training signal, without requiring a global gradient computation. Both approaches demonstrate that alternatives to backpropagation can achieve competitive performance, though through very different mechanisms.

12.3 Energy Analysis: The Case for Integer-Ordinal Hardware

The preceding discussion frames ArrowFlow as an alternative computational paradigm. But *why* would anyone prefer integer comparisons over floating-point multiply-accumulates? The answer is energy: ArrowFlow’s core sort layers use *no floating-point arithmetic*. Every operation—displacement computation, distance accumulation, filter updates, output ranking—is an integer comparison, subtraction, absolute value, or addition. This section quantifies the energy implications by comparing ArrowFlow’s operation profile against a standard MLP on a per-layer and per-inference basis.

Operation counting. Consider a sort layer with N filters over a vocabulary of size V . The forward pass for one data point performs:

1. **Index table construction:** Build a lookup from input items to positions. With a pre-allocated index array, this is V integer writes.
2. **Displacement computation:** For each of N filters, look up the input position of each of V items (one integer table read), compute the signed displacement (one integer subtraction), take the absolute value, and add to a running sum. Total: $\sim 3NV$ integer operations.
3. **Output ranking:** Arg-sort the N distance values, requiring $O(N \log N)$ integer comparisons.

The total is approximately $3NV + N \log_2 N$ integer operations per layer per data point. No multiply-accumulate (MAC) operations appear anywhere.

A comparable MLP layer (V inputs, N outputs) performs NV floating-point MACs (each a multiply plus an add), N bias additions, and N activation evaluations: approximately NV FP32 MACs total.

Energy per operation. Horowitz’s widely cited ISSCC 2014 analysis [Horowitz, 2014] established the energy cost of arithmetic operations at 45nm CMOS, reproduced by Sze et al. [Sze et al., 2017] in the context of DNN accelerator design:

Table 27: Energy per operation at 45nm CMOS [Horowitz, 2014]. ArrowFlow uses only operations in the top group; standard neural networks use operations in both groups.

Operation	Energy (pJ)	Used by
8-bit integer ADD	0.03	ArrowFlow
32-bit integer ADD/CMP	0.1	ArrowFlow
32-bit integer MUL	3.1	—
32-bit FP ADD	0.9	MLP
32-bit FP MUL	3.7	MLP
32-bit FP MAC	4.6	MLP
32-bit SRAM read (8KB)	5	both
32-bit DRAM read	640	both

A single FP32 MAC costs 4.6 pJ; a single integer comparison or addition costs 0.1 pJ—a **46 $\times$ gap** at the arithmetic level. If ArrowFlow’s permutation indices fit in 8 bits ($V \leq 256$, which covers all configurations in this paper), the relevant integer operations cost 0.03 pJ, widening the gap to **$\sim 150\times$** .

Per-layer energy comparison. For a concrete comparison, consider ArrowFlow’s [128] configuration on Digits ($V = 64$, $N = 128$) versus an MLP with a $64 \rightarrow 128$ hidden layer:

Table 28: Energy per layer, single data point. ArrowFlow [128] ($V=64$, $N=128$) vs. MLP ($64 \rightarrow 128$). All energies at 45nm CMOS.

	Operations	Energy (pJ)	Op type
ArrowFlow sort layer			
Displacement ($3NV$)	24,576	2,458	INT32 @ 0.1 pJ
Argsort ($N \log_2 N$)	896	90	INT32 @ 0.1 pJ
<i>Total</i>	25,472	2,547	
MLP layer			
Matrix multiply (NV MACs)	8,192	37,683	FP32 @ 4.6 pJ
Bias + ReLU	256	230	FP32
<i>Total</i>	8,448	37,914	
Ratio	14.9$\times$ in favor of ArrowFlow		

ArrowFlow performs $\sim 3\times$ more operations but each costs $\sim 46\times$ less energy, yielding a net **$\sim 15\times$ arithmetic energy advantage** per layer.

Memory access energy. In practice, memory access dominates arithmetic cost in neural network inference [Sze et al., 2017]. ArrowFlow’s parameters are more compact: each filter is a permutation of V items stored as 8-bit indices, requiring NV bytes per layer. An MLP weight matrix requires $4NV$ bytes (FP32). For $N = 128$, $V = 64$:

- ArrowFlow filter bank: $128 \times 64 = 8,192$ bytes (fits in 8KB SRAM)

- MLP weight matrix: $128 \times 64 \times 4 = 32,768$ bytes (requires 32KB)

When parameters fit in SRAM (5 pJ per 32-bit read), the total memory energy for reading all weights once is 40,960 pJ for the MLP vs. $\sim 10,240$ pJ for ArrowFlow’s 8-bit indices—a $4\times$ reduction. If parameters spill to DRAM (640 pJ per read), the memory advantage becomes the dominant factor.

Full inference comparison. Table 29 compares full-inference energy for ArrowFlow’s best Digits configuration (7-view ensemble, [256] layers, $V = 64$) against a single MLP ($64 \rightarrow 128 \rightarrow 10$):

Table 29: Full inference energy comparison on Digits. ArrowFlow: 7-view [256], $V=64$. MLP: $64 \rightarrow 128 \rightarrow 10$. Arithmetic energy only (45nm CMOS).

Component	ArrowFlow (pJ)	MLP (pJ)
Hidden, dist.+sort ($\times 7$)	$7(49,152+2,048)(0.1) = 35,840$	$8,192(4.6) = 37,683$
Output, dist.+arg min ($\times 7$)	$7 \times 7,690(0.1) = 5,383$	$1,280(4.6) = 5,888$
Majority vote	~ 70	—
Total arithmetic	41,293	43,571
Ratio	$\approx 1.05\times$ in favor of ArrowFlow	

Here the output layer compares the length-256 hidden ranking to $C = 10$ class filters ($3CV = 7,680$ displacement operations per view) and predicts by arg min rather than a full sort; the hidden layer includes its $N \log_2 N$ ranking cost. The 7-view ensemble erodes nearly all of the per-operation advantage, reducing the ratio to only $\approx 1.05\times$ at the full-inference arithmetic level. However, three factors make the real-world advantage substantially larger:

1. **Memory bandwidth:** ArrowFlow’s 8-bit parameters require $4\times$ less memory bandwidth than FP32 weights. On memory-bound hardware (edge devices, neuromorphic chips), this translates directly to energy savings.
2. **Parallelism:** The 7 views are embarrassingly parallel and operate on compact integer data, making them highly amenable to SIMD or multi-core execution with minimal inter-core communication.
3. **No multiply hardware:** ArrowFlow requires only comparators, adders, and index tables—no floating-point multiply units. On an ASIC designed for ArrowFlow’s operation profile, the die area and static power savings from eliminating FP multipliers would be substantial.

Neuromorphic hardware alignment. ArrowFlow’s computation maps naturally to neuromorphic architectures:

- **Rank-order coding.** Thorpe’s rank-order coding [Thorpe and Gautrais, 1998, Thorpe et al., 2001] encodes stimulus strength via the *order* of neural spikes rather than firing rates—precisely the representation ArrowFlow operates on. A ranking filter in

ArrowFlow corresponds to a stored spike-order template, and the footrule distance corresponds to the total spike-timing displacement. Imam and Cleland [Imam and Cleland, 2020] demonstrated that temporal-order-based classification on Intel Loihi achieves $1000\times$ better energy efficiency than conventional CPU solutions.

- **Operation mapping.** ArrowFlow’s displacement computation (integer subtraction + absolute value) maps to spike-timing difference circuits. The argsort operation maps to a sorting network—a fixed-topology circuit of compare-and-swap units. Batcher’s bitonic sorting networks [Batcher, 1968] require $O(N \log^2 N)$ comparators with a completely data-independent wiring pattern, ideal for fixed silicon. Each compare-and-swap costs ~ 0.15 pJ at 45nm, so sorting $N = 128$ distances requires ~ 100 pJ total—negligible compared to the displacement computation.
- **Event-driven sparsity.** Neuromorphic chips like Intel Loihi [Davies et al., 2018] (23.6 pJ/op at 14nm) and IBM TrueNorth [Merolla et al., 2014] (26 pJ/event at 28nm) achieve low energy through event-driven computation: neurons consume energy only when they spike. ArrowFlow’s winner-take-all dynamics (Section 4, ND violation) create natural sparsity—most filters have large distances and only the closest few matter. An event-driven implementation could skip distance computation once a filter’s partial distance exceeds the current minimum, reducing average-case energy well below the worst-case $3NV$ operations.

Scope and caveats. This analysis is deliberately narrow: it counts *arithmetic* energy in the core sort layers only. Four caveats bound any system-level claim. (i) The *encoder* is not free—polynomial expansion, standardization, and especially the random/LDA projection use floating-point multiply-adds unless the projections are precomputed, quantized, or replaced by sparse/sign projections; our accounting excludes these. (ii) A fair deployment baseline is often an *INT8-quantized* MLP, not FP32, which narrows the per-operation gap substantially. (iii) Classification may not require sorting all N distances—nearest-filter or top- k selection would lower ArrowFlow’s cost further, but the comparison should hold the operation set fixed. (iv) Memory access typically dominates arithmetic, so the realized advantage depends on the memory hierarchy and the degree of view parallelism. We therefore present the figures as an arithmetic-level indication, not a measured system-level speedup.

Summary. At the arithmetic-operation level, ArrowFlow’s core layers replace FP32 MACs with integer comparisons and additions, a $15\text{--}150\times$ per-operation gap that narrows at the full-inference level (to $\approx 1.05\times$ for the seven-view Digits configuration). The system-level advantage will depend on the encoder, memory hierarchy, quantization baseline, and view parallelism. Even so, the combination of compact 8-bit parameters, no floating-point multiply units, and natural alignment with neuromorphic spike-order coding makes a custom ASIC or neuromorphic implementation of ArrowFlow’s sort layers—comparators, integer adders, and index tables—a plausible route to low-energy inference, which we leave to future hardware work.

12.4 The Argsort Bottleneck: A Fundamental Limit

The argsort encoding is simultaneously ArrowFlow’s greatest strength and its most fundamental limitation. By converting real-valued features to orderings, ArrowFlow gains scale invariance, noise robustness (Theorem 4), and rank-only (magnitude-hiding) operation—but loses all magnitude information. Theorem 7 quantifies this: the length- e ordinal code has capacity $\log_2(e!) \approx e \log_2 e$ bits, finite and far below the infinite capacity of real-valued representations. Polynomial expansion reshapes the feature geometry feeding this fixed-length code rather than enlarging its capacity, and by Proposition 8 it amplifies noise by a factor of B^{k-1} , weakening the stability bound. This tension—more information capacity vs. less noise robustness—is the formal basis of the polynomial trade-off documented in Section 11.8.

12.5 Limitations

- **Computational cost:** Each forward pass is $O(NV)$ per data point. While the forward pass has been vectorized via batch distance computation, the backward pass remains sequential. This makes ArrowFlow approximately 10× slower than an equivalently-sized MLP per training iteration.
- **Information loss:** The argsort encoding discards magnitude information. Polynomial expansion mitigates but does not eliminate this, and introduces a robustness trade-off.
- **Scaling gap:** On larger datasets, ArrowFlow lags behind gradient-based methods. On MNIST (Section 11.7), best ArrowFlow achieves 9.1% vs. MLP’s 4.2%—a persistent ~5pp gap. However, the gap narrows with wider architectures and has not plateaued, suggesting limited capacity is at least one bottleneck (though not necessarily the only one).
- **Natively ordinal data:** ArrowFlow’s whole-permutation distance is too holistic when only local ordinal features are predictive (Sushi experiment, Section 11.5).
- **Rank-transform baseline:** On gene expression data (Section 11.6), SVM on rank-transformed features achieves comparable or better batch-effect robustness than ArrowFlow native. The robustness advantage belongs to the ordinal representation principle, not to ArrowFlow specifically—though ArrowFlow provides this guarantee architecturally rather than as a preprocessing choice.
- **Hybrid architecture failure:** The tensor→sort interface (Section 9) destroys continuous representations via argsort discretization, preventing effective hybrid architectures.
- **Slow convergence:** The permutation-matrix accumulation makes conservative updates, requiring 200–500 iterations vs. ~50 epochs for gradient-based methods.

12.6 Future Directions

1. **Batch vectorization:** The forward and backward passes process data points sequentially. Vectorizing via `scipy.cdist` or custom CUDA kernels could provide 10–50× speedup, making ArrowFlow practical for larger datasets.

2. **Learned projections:** Replace random projection with end-to-end learned projections that minimize encoding loss while maintaining ensemble diversity. This could bridge the argsort bottleneck without abandoning the ordinal architecture.
3. **Soft-sort encoding:** Using differentiable argsort approximations [Prillo and Eisen-schlos, 2020] to preserve partial magnitude information, potentially closing the gap on magnitude-sensitive datasets like Wine Quality.
4. **Position-wise attention:** For natively ordinal data, a mechanism that weights individual item positions (rather than whole-permutation distances) could address the holistic-distance limitation exposed by the Sushi experiment.
5. **Neuromorphic implementation:** ArrowFlow’s integer-only arithmetic is $15\text{--}150\times$ cheaper per operation than FP32 MACs (Section 12.3). A custom ASIC implementing sort layers as comparator arrays and index tables could make inference dramatically cheaper than conventional networks, particularly on neuromorphic platforms [Davies et al., 2018, Merolla et al., 2014] where spike-order coding is native.
6. **Mode selection framework:** The polynomial trade-off (Section 11.8) suggests a practical deployment strategy: automatically select $\text{pol_deg}=1$ when the data is noisy, privacy-constrained, or incomplete; $\text{pol_deg} > 1$ when clean accuracy is paramount.
7. **Text and sequence domains:** The CountVectorizer nonzero encoding (converting word presence to native permutations) opens a path for applying ArrowFlow to text classification without the projection pipeline. Initial IMDB experiments are underway.
8. **Broader combinatorial primitives:** Displacement-accumulation generalizes beyond permutations to other discrete structures—matchings, partial orders, lattice elements—potentially yielding a family of combinatorial learning architectures.

13 Conclusion

ArrowFlow demonstrates that machine learning can operate effectively in the space of permutations. The architecture processes information through ranking filters—discrete permutations learned via motion accumulation rather than gradient descent—and achieves competitive classification accuracy through multi-view ensembles of diverse ordinal projections.

The key empirical findings, across 7 UCI datasets, MNIST, gene expression cancer classification, and preference data, with GridSearchCV-tuned baselines, are:

1. ArrowFlow **matches or slightly edges every tuned baseline on Iris** (2.7% vs. 3.3%; a sub-sample margin, see Section 10) and is competitive on 5 of 7 UCI datasets, showing that zero-floating-point-parameter architectures can rival gradient-trained networks.
2. The **polynomial degree acts as a master switch** between accuracy and robustness. At degree 1, ArrowFlow provides measurable noise robustness (8–28% less degradation), near-zero-cost rank-only (magnitude-hiding) operation (+0.5pp under rank transforms), and superior missing-feature handling (lowest error on Iris at 50% masking). At higher degrees, clean accuracy improves but perturbation amplification erases these advantages.

3. On gene expression data, ordinal encoding provides **exact invariance to within-sample monotone batch effects**: SVM on raw values collapses from 0.6% to 82.6% under log, $\sqrt{\cdot}$, or global scaling, while rank-based methods remain stable at 2.5%. Under realistic per-gene batch effects ($\sigma \leq 0.5$), rank methods are unaffected while raw-input SVM error increases $28\times$. ArrowFlow embodies this ordinal robustness architecturally.
4. On **MNIST via PCA**, ArrowFlow achieves 9.1% error using only ordinal comparisons. It **scales strongly** with width and depth (18.7% $\rightarrow$ 9.1%), and the scaling curve has not plateaued, suggesting capacity is the bottleneck.
5. The **multi-view ensemble** with diverse projections is the single most impactful component, reducing error by up to $\sim 3\times$ (1.3–3.3 $\times$ across datasets).
6. **Freezing the output layer** consistently improves performance, a novel training strategy analogous to fixing classifier heads in transfer learning.
7. ArrowFlow’s **learning genuinely helps**: 2.5–12.5 $\times$ improvement over kNN on the same encoded features proves that filter updates learn discriminative ordinal structure.

Beyond empirics, the theoretical analysis (Section 8) provides formal grounding: the argsort stability theorem (Theorem 4) and its Gaussian corollary explain the noise robustness; the information capacity theorem (Theorem 7) quantifies the encoding bottleneck at $\log_2(e!)$ bits for the e -dimensional projected code; the polynomial noise amplification result (Proposition 8) explains the accuracy–robustness trade-off; and the accumulation consistency result (Proposition 9) guarantees that the learning rule converges to the mean-position ranking (the Borda aggregate of accumulated evidence).

The deeper contribution is conceptual: ArrowFlow shows that the violations of Arrow’s social-choice axioms—context dependence, specialization, symmetry breaking—are not obstacles but *mechanisms* for learning. Each ranking layer is a micro-democracy of filters whose deliberate “unfairness” creates nonlinearity, sparsity, and hierarchical composition. This reframes neural computation through the lens of combinatorics and voting theory, opening a new research direction at the intersection of social choice, rank-order coding, and representation learning.

ArrowFlow is an early entry in ordinal machine learning, not a finished one. The encoding bottleneck, the scaling gap, and the holistic-distance limitation are real constraints that bound the current system’s practical applicability. But the fact that a system with *zero floating-point parameters* in its core layers can reach 2.7% error on Iris, 9.1% on MNIST, and exact invariance to monotone batch effects on gene expression data—and that we can *partly explain*, with proofs tied to the encoding and the learning rule, why its robustness properties hold—is, we believe, a foundation worth building on.

References

Dimitris Achlioptas. Database-friendly random projections: Johnson-lindenstrauss with binary coins. *Journal of Computer and System Sciences*, 66(4):671–687, 2003.

- Rosa I Arriaga and Santosh Vempala. An algorithmic theory of learning: Robust concepts and random projection. *Machine Learning*, 63(2):161–182, 2006.
- Kenneth J Arrow. *Social Choice and Individual Values*. John Wiley & Sons, 1951.
- Kenneth E Batcher. Sorting networks and their applications. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pages 307–314, 1968.
- Paul Bertens. Rank ordered autoencoders. *arXiv preprint arXiv:1605.01749*, 2016.
- Mathieu Blondel, Olivier Teboul, Quentin Berthet, and Josip Djolonga. Fast differentiable sorting and ranking. In *International Conference on Machine Learning*, pages 950–959. PMLR, 2020.
- Lina Bonilla, Jacques Gautrais, Simon Thorpe, and Timothée Masquelier. Analyzing time-to-first-spike coding schemes: A theoretical approach. *Frontiers in Neuroscience*, 16: 971937, 2022.
- Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.
- Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. Learning to rank using gradient descent. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 89–96. ACM, 2005.
- Christopher JC Burges. From ranknet to lambdarank to lambdamart: An overview. *Technical Report MSR-TR-2010-82, Microsoft Research*, 2010.
- Timothy I Cannings and Richard J Samworth. Random-projection ensemble classification. *Journal of the Royal Statistical Society: Series B*, 79(4):959–1035, 2017.
- Vincent Conitzer and Tuomas Sandholm. Common voting rules as maximum likelihood estimators. In *Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence*, pages 145–152, 2005.
- W Jay Conover and Ronald L Iman. Rank transformations as a bridge between parametric and nonparametric statistics. *The American Statistician*, 35(3):124–129, 1981.
- Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham Chinya, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1):82–99, 2018.
- Persi Diaconis and Ronald Graham. Spearman’s footrule as a measure of disarray. *Journal of the Royal Statistical Society. Series B*, 1977.
- Thomas G Dietterich. Ensemble methods in machine learning. *Multiple Classifier Systems*, 1857:1–15, 2000.
- Cynthia Dwork, Ravi Kumar, Moni Naor, and D Sivakumar. Rank aggregation methods for the web. In *Proceedings of the 10th International Conference on World Wide Web*, pages 613–622. ACM, 2001.

- Ronald Fagin, Ravi Kumar, and Dandapani Sivakumar. Comparing top k lists. *SIAM Journal on Discrete Mathematics*, 2003.
- Jacques Gautrais and Simon Thorpe. Rate coding versus temporal order coding: a theoretical approach. *BioSystems*, 48(1-3):57–65, 1998.
- Allan Gibbard. Manipulation of voting schemes: A general result. *Econometrica*, 41(4):587–601, 1973.
- Stephen Grossberg. Competitive learning: From interactive activation to adaptive resonance. *Cognitive Science*, 11(1):23–63, 1987.
- Aditya Grover, Eric Wang, Aaron Zweig, and Stefano Ermon. Stochastic optimization of sorting networks via continuous relaxations. In *ICLR*, 2019.
- Geoffrey Hinton. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2022.
- Mark Horowitz. 1.1 computing’s energy problem (and what we can do about it). In *IEEE International Solid-State Circuits Conference (ISSCC)*, pages 10–14. IEEE, 2014.
- Nabil Imam and Thomas A Cleland. Rapid online learning and robust recall in a neuromorphic olfactory circuit. *Nature Machine Intelligence*, 2:181–191, 2020.
- Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018.
- William B Johnson and Joram Lindenstrauss. Extensions of lipschitz mappings into a hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.
- Jérôme Jost, Dimitri Ognibene, and Charles Spence. Rank-order coding and timing variability reduction. *Network: Computation in Neural Systems*, 13(1):55–70, 2002.
- Maurice Kendall. A new measure of rank correlation. *Biometrika*, 1938.
- Teuvo Kohonen. The self-organizing map. *Proceedings of the IEEE*, 78(9):1464–1480, 1990.
- Dong-Hyun Lee, Saizheng Zhang, Asja Fischer, and Yoshua Bengio. Difference target propagation. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 498–515. Springer, 2015.
- Colin L Mallows. Non-null ranking models. i. *Biometrika*, 44(1/2):114–130, 1957.
- Gonzalo Mena, David Belanger, Scott Linderman, and Jasper Snoek. Learning latent permutations with gumbel-sinkhorn networks. In *ICLR*, 2018.

- Paul A Merolla, John V Arthur, Rodrigo Alvarez-Icaza, Andrew S Cassidy, Jun Sawada, Filipp Akopyan, Bryan L Jackson, Nabil Imam, Chen Guo, Yutaka Nakamura, et al. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345(6197):668–673, 2014.
- Sebastian Prillo and Julian Eisenschlos. Softsort: A continuous relaxation for the argsort operator. In *ICML*, 2020.
- Mark Allen Satterthwaite. Strategy-proofness and arrow’s conditions: Existence and correspondence theorems for voting procedures and social welfare functions. *Journal of Economic Theory*, 10(2):187–217, 1975.
- Robert E Schapire. The strength of weak learnability. *Machine Learning*, 5(2):197–227, 1990.
- Richard Sinkhorn and Paul Knopp. Diagonal equivalence to matrices with prescribed row and column sums. *The American Mathematical Monthly*, 74(4):402–405, 1967.
- Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S Emer. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, 105(12):2295–2329, 2017.
- Simon Thorpe and Jacques Gautrais. Rapid visual processing using spike asynchrony. *Neural Computation*, 10(2):281–288, 1998.
- Simon J Thorpe, Arnaud Delorme, and Rufin VanRullen. Spike-based strategies for rapid processing. *Neural Networks*, 14(6-7):715–726, 2001.
- Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, 2017.
- Oriol Vinyals, Samy Bengio, and Manjunath Kudlur. Order matters: Sequence to sequence for sets. *arXiv preprint arXiv:1511.06391*, 2015a.
- Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *NeurIPS*, 2015b.
- Chang Xu, Dacheng Tao, and Chao Xu. A survey on multi-view learning. *arXiv preprint arXiv:1304.5634*, 2013.